



TECNOLOGICO NACIONAL DE MEXICO

INSTITUTO TECNOLÓGICO DE TUXTEPEC

Actividad:

Curso Intensivo

Materia:

Ciencia de Datos

Carrera:

Ing. Sistemas Computacionales

Alumna:

Guerrero Vásquez Fernanda Ximena

Docente:

Dr. Julio Aguilar Carmona

Tuxtepec, Oax.

5 de mayo del 2026

INTRODUCCION

The screenshot shows the Kaggle website with the URL <https://www.kaggle.com/learn-guide/5-day-agents>. The page features a sidebar with navigation links: Inicio, Competiciones, Referencias, Arena de Juego, Centro de datos, and Más. The main content area is titled 'Curso intensivo de agentes de IA de 5 días con Google'. It includes a search bar, login and registration buttons, and a detailed introduction to the course. The introduction explains that the course is a practical program originally held from November 10 to 14, 2025, now available as a Kaggle Learn guide. It was developed by Google ML researchers to help developers explore the fundamentals and practical applications of AI agents. The course covers basic components like models, tools, orchestration, memory, and evaluation, and aims to move beyond LLM prototypes to production-ready systems. It combines deep conceptual analysis with practical examples, code labs, and live debates. The page concludes with a welcome message to the 'Guía de Aprendizaje Intensivo para Agentes!' and a section titled '¿Qué se está cubriendo?'.

Curso intensivo de agentes de IA de 5 días con Google

¿Qué es el Intensivo de Agentes de IA de 5 días?

El curso intensivo de agentes de IA de 5 días con Google es un programa práctico que originalmente se celebró en directo del 10 al 14 de noviembre de 2025. Ahora está disponible como una guía Kaggle Learn autodidacta para que cualquiera pueda explorar los fundamentos, la arquitectura y el desarrollo práctico de los agentes de IA.

Este curso fue elaborado por los investigadores e ingenieros de ML de Google para ayudar a los desarrolladores a explorar los fundamentos y aplicaciones prácticas de los agentes de IA. Aprenderás los componentes básicos: modelos, herramientas, orquestación, memoria y evaluación. Por último, descubrirás cómo los agentes van más allá de los prototipos LLM para convertirse en sistemas listos para producción.

Cada día combina análisis conceptuales profundos con ejemplos prácticos, laboratorios de código y debates en vivo. Al final, estarás listo para construir, evaluar y desplegar agentes que resuelvan problemas del mundo real.

¡Bienvenido a nuestra Guía de Aprendizaje Intensivo para Agentes!

¿Qué se está cubriendo?

KAGGLE

The screenshot shows the Kaggle website with the URL <https://www.kaggle.com>. The page features a sidebar with navigation links: Competiciones, Referencias, Arena de Juego, Centro de datos, and more. The main content area is titled 'El Campo de Pruebas de IA del Mundo'. It includes a search bar, login and registration buttons, and a detailed introduction to the competition. The introduction explains that the competition is a collaborative effort to evaluate AI agents, models, and technology through benchmarks, competitions, and hackathons. It encourages users to join 30M+ builders, researchers, and labs. The page includes a 'Regístrate en Google' button and a 'Regístrate por correo electrónico' link. Below the main content, there are three sections: 'Constructores de IA' (builders), 'Organizaciones' (organizations), and 'Investigadores' (researchers), each with a brief description and an icon. The page concludes with a footer showing the current temperature (29°C) and the date (4/28/2026).

El Campo de Pruebas de IA del Mundo

Descubre qué funciona realmente en IA. Únete a 30M+ constructores, investigadores y laboratorios que evalúan agentes, modelos y tecnología de vanguardia a través de benchmarks, competencias y hackathons colaborativos.

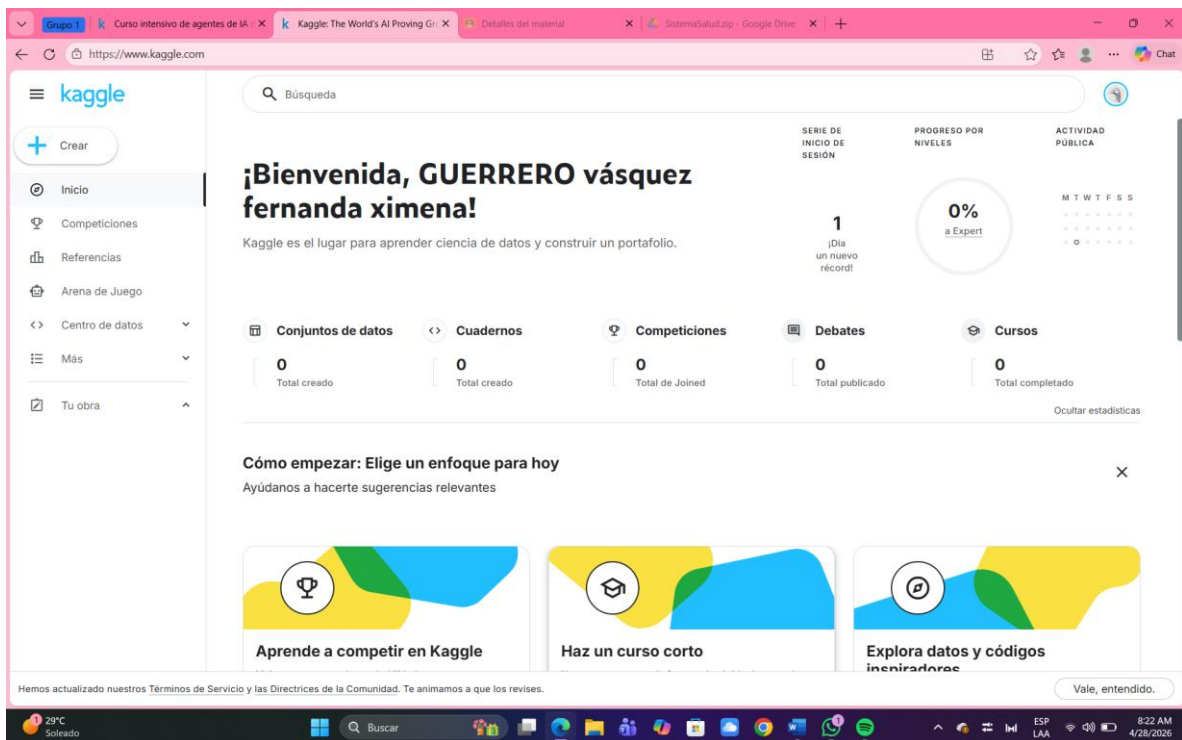
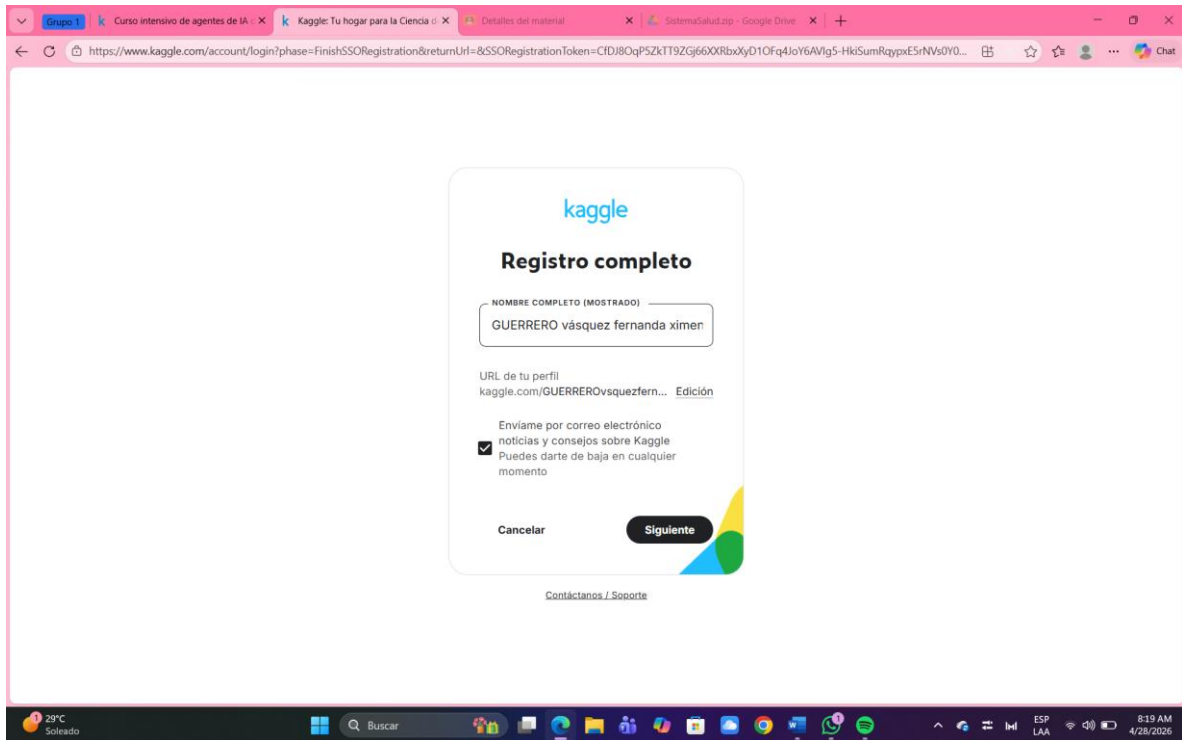
[Regístrate en Google](#) [Regístrate por correo electrónico](#)

¿Quién está en Kaggle?

Constructores de IA
Pon a prueba tus habilidades de IA contra las mejores del mundo. Compíte, aprende y crea un portafolio.

Organizaciones
Llevar desafíos fronterizos a millones para revelar el verdadero estado del arte.

Investigadores
Desarrollar, publicar y mantener estándares rigurosos en una plataforma abierta.



Grupos 1 k Curso intensivo de agentes de IA x (153) How to use Kaggle Not... x Kaggle: The World's AI Proving Gr... x Detalles del material x SistemaSalud.zip - Google Drive x + - Chat

https://www.youtube.com/watch?v=BPQDZLptgDc

YouTube

Buscar

+ Crear

kaggle 5-Day AI Agents Intensive Course

Resumir

Thank you!



107 k vistas · hace 3 semanas

Welcome to Kaggle's 5-Day AI Agents Intensive course with Google!

29°C Soleado

Ahora con 55% de Descuento

Comprar ahora

Todos De Kaggle Aprendizajes automáticos

KAGGLE

¿QUIERES COMO EMPEZAR?

9:47

1.3 K vistas · hace 8 meses

Pero, ¿qué es un GPT?

Introducción visual a los ...

3Blue1Brown

10 M de vistas · hace 2 años

27:14

Doblado

Soaking Worship Music // Peace

Moments of Worship

2.3 M de vistas · hace 2 años

1:19:23

Ariana Grande & Justin Bieber - Stuck with U

Ariana Grande

268 M de vistas · hace 5 años

4:18

Para recaudar fondos

vevo

8:27 AM 4/28/2026

Grupos 1 k Curso intensivo de agentes de IA x Persona x Kaggle: The World's AI Proving Gr... x Detalles del material x SistemaSalud.zip - Google Drive x + - Chat

https://kaggle.withpersona.com/verify?inquiry-id=inq_A7UTz9KMHk6fiv9Q4NQ8E28a/Uje7t

¡Felicitaciones, ya terminaste!

Gracias por verificar tu identidad.

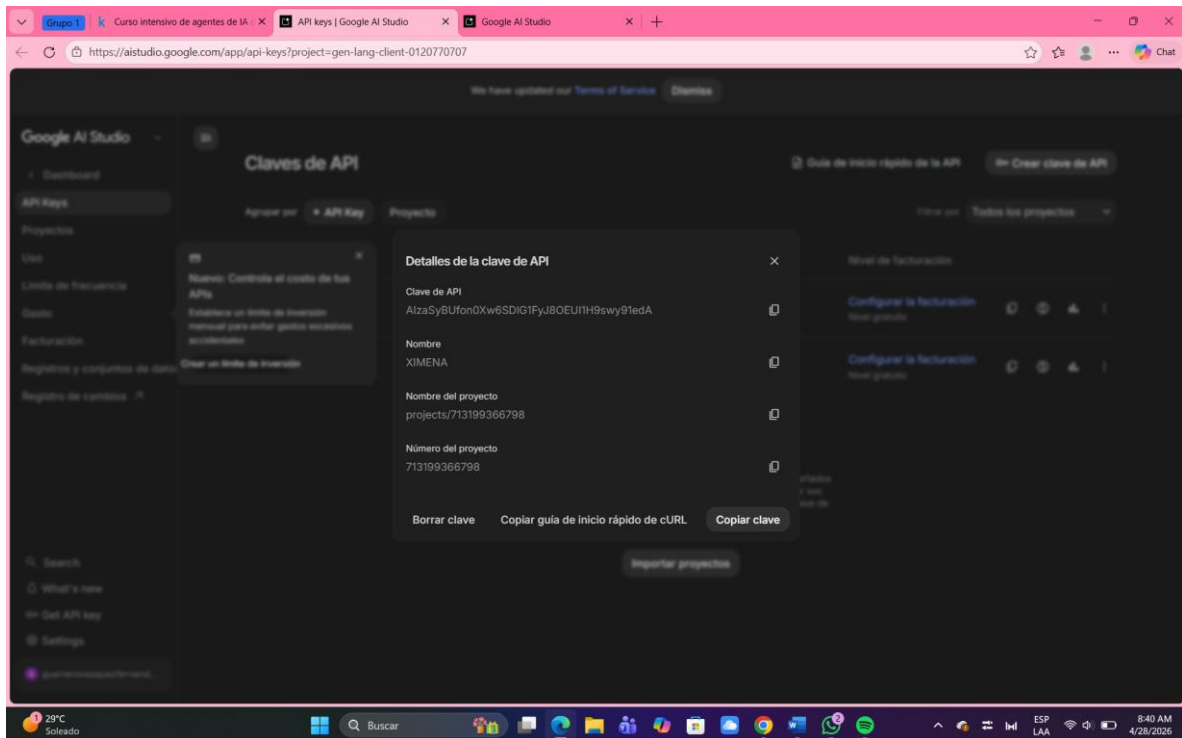
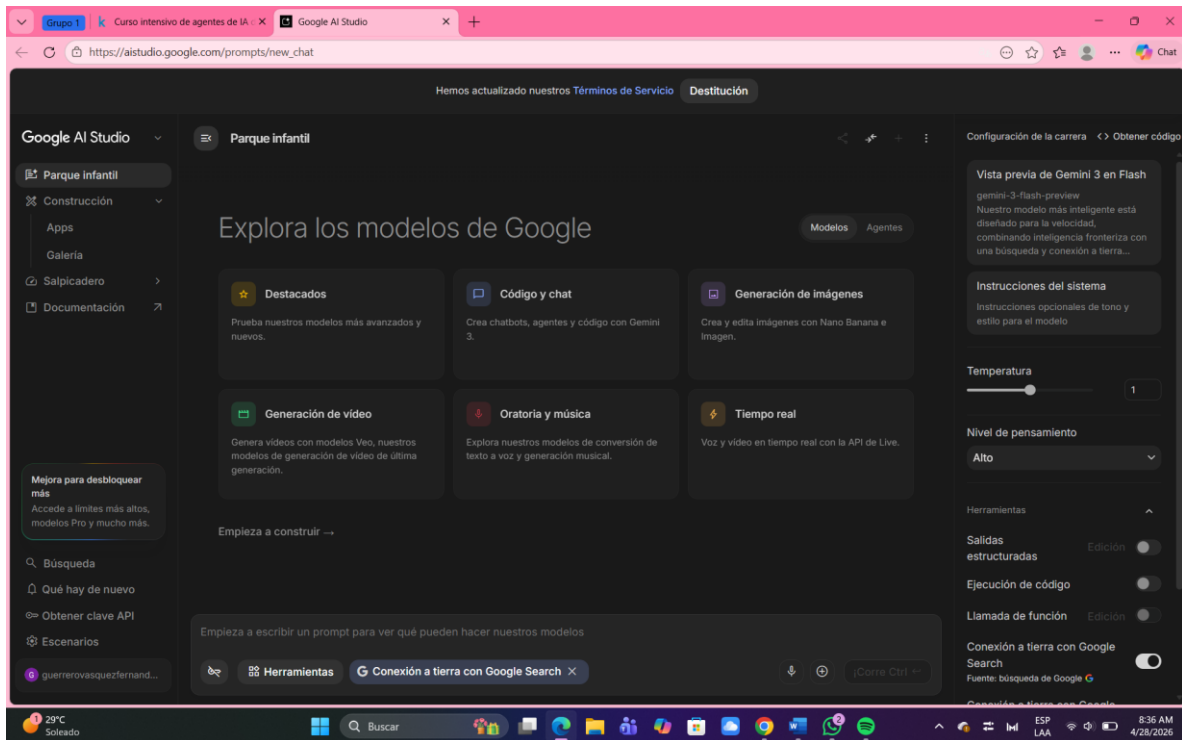


Hecho

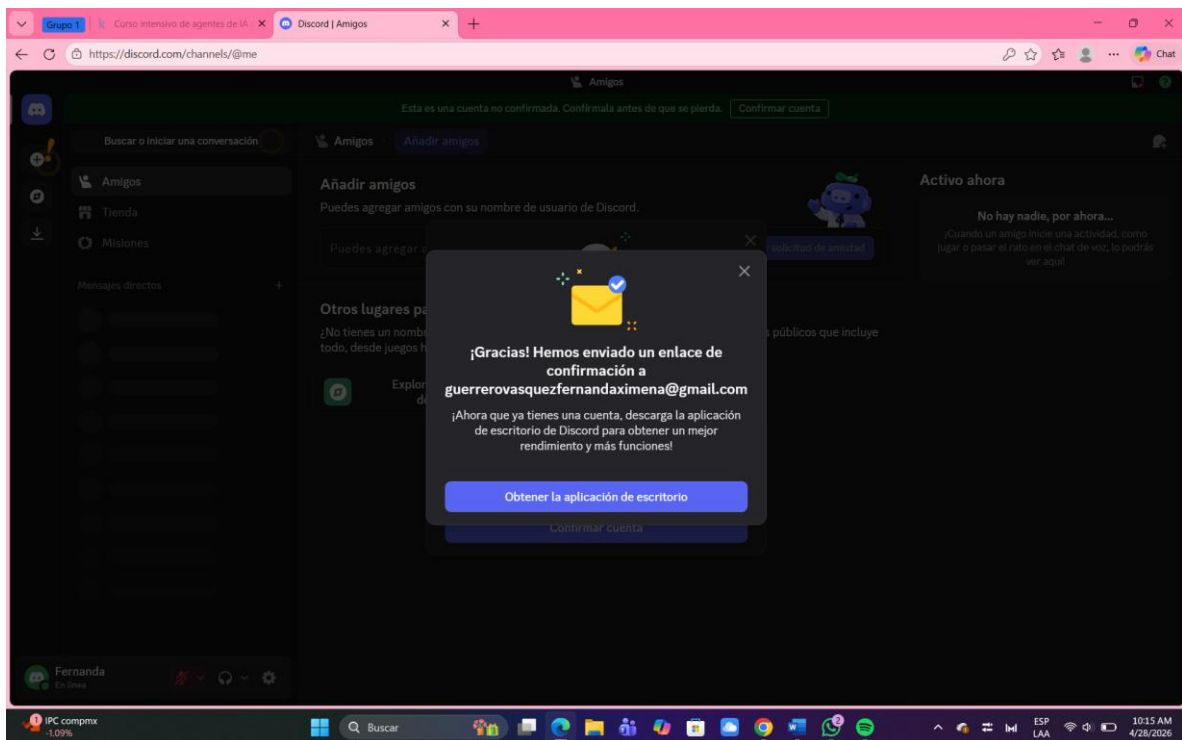
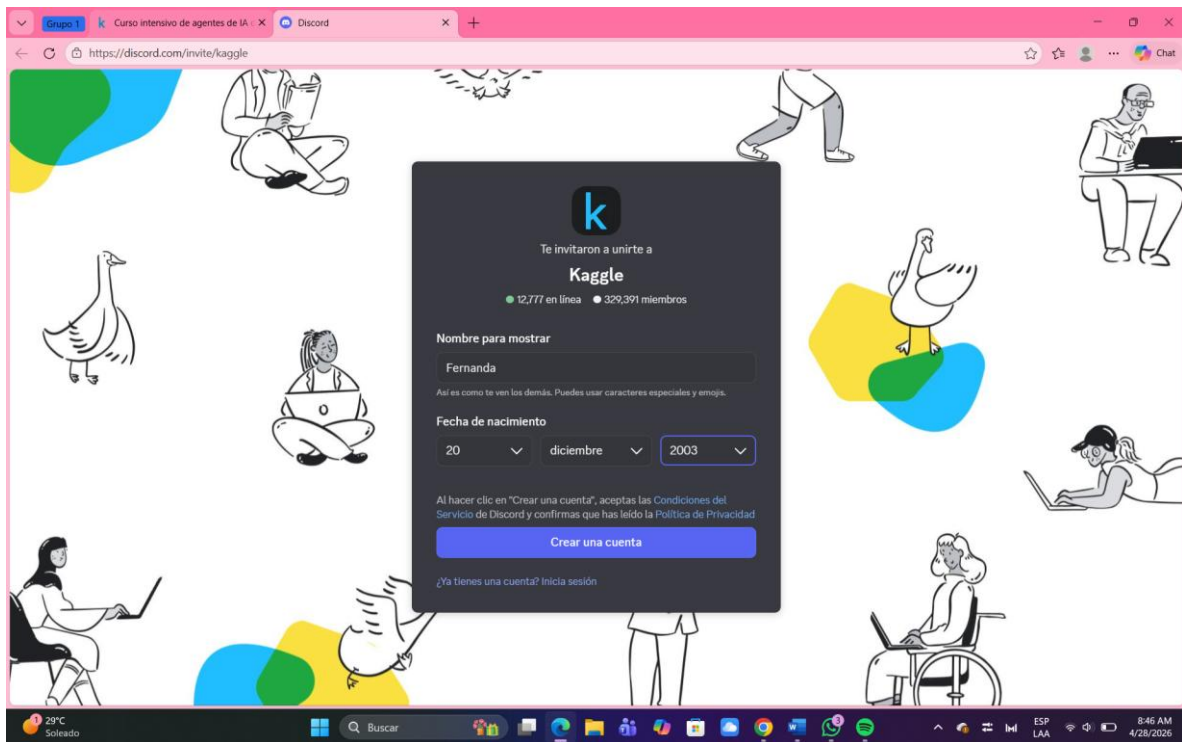
29°C Soleado

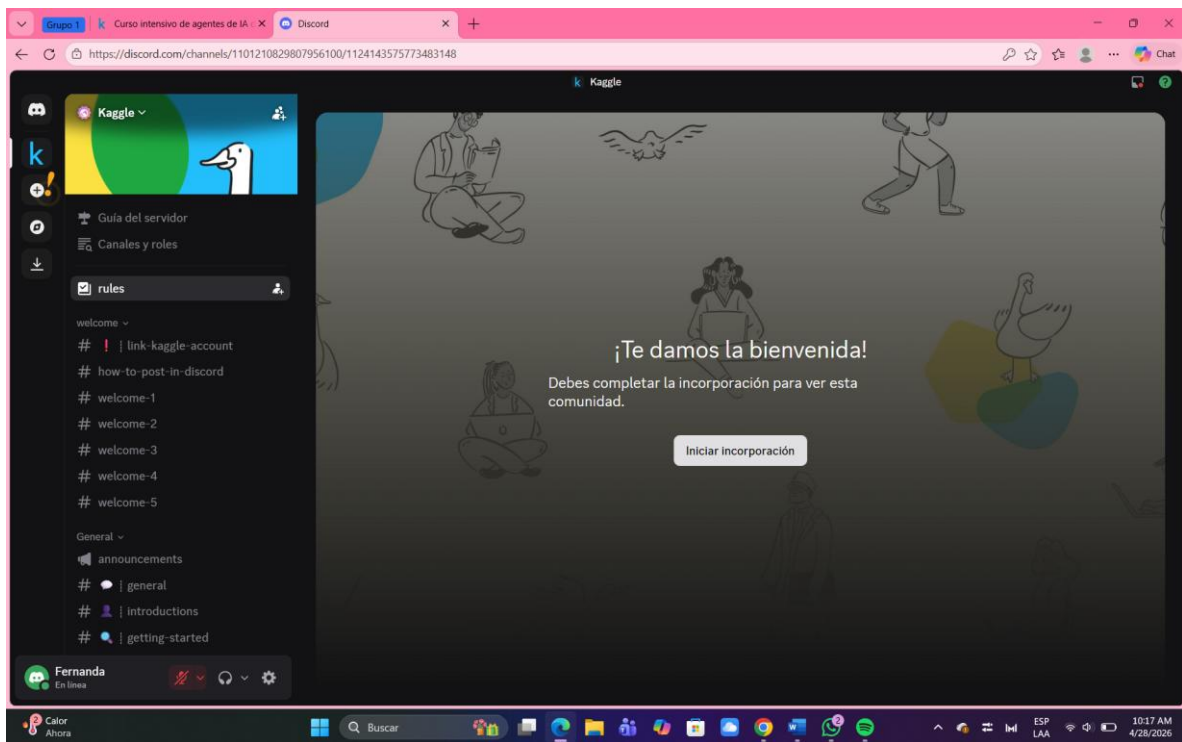
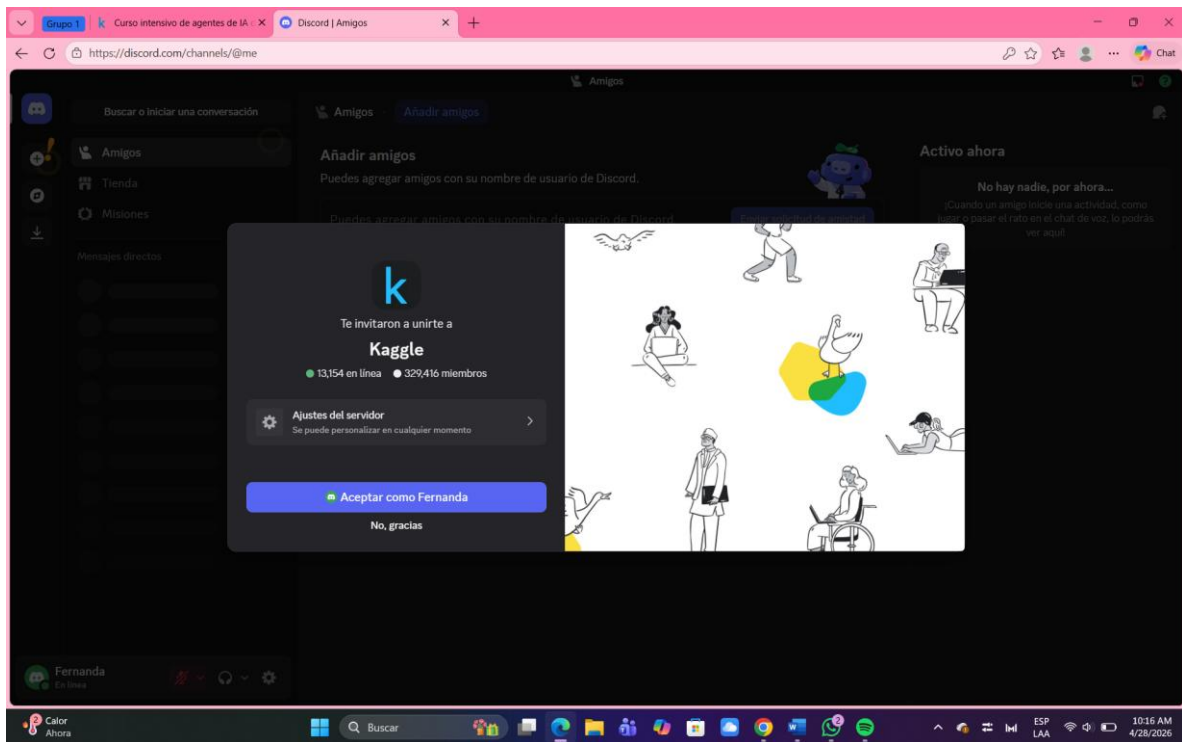
8:28 AM 4/28/2026

IA studio



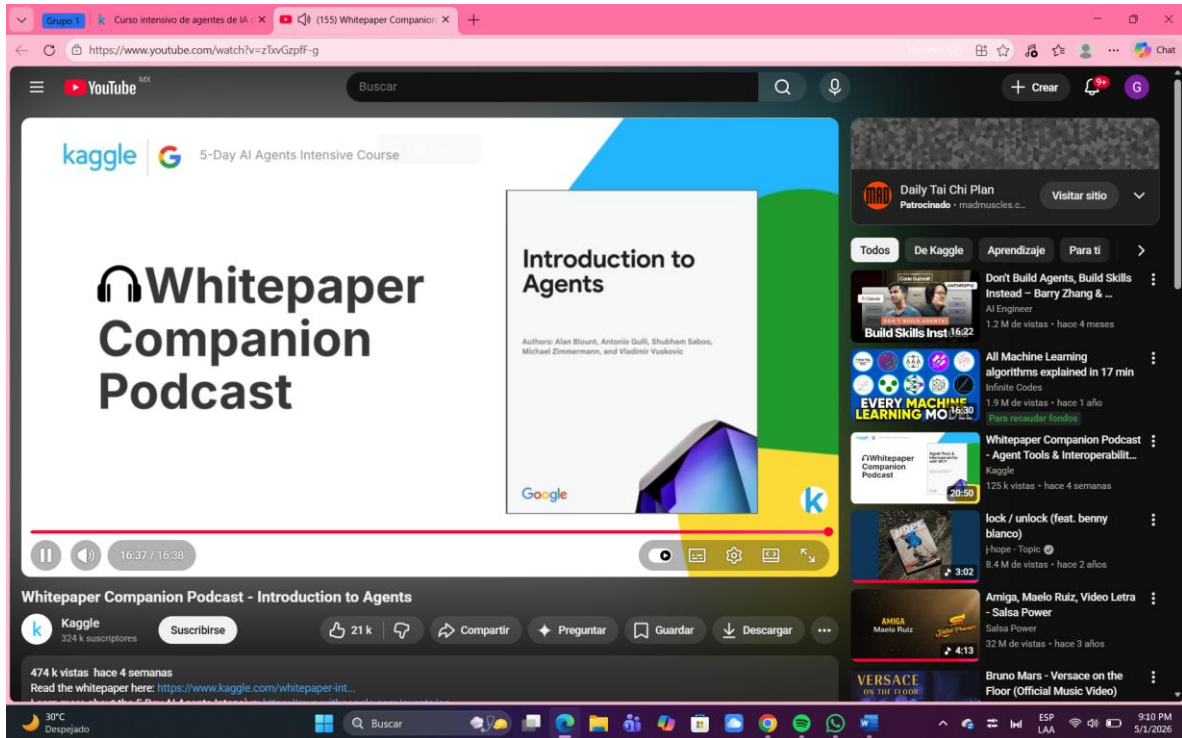
DISCORD DE KAGGLE





Día 1 (Introducción a los agentes)

Escucha el episodio resumen del podcast de esta unidad.



Para complementar el podcast, lee el whitepaper

The screenshot shows the Kaggle website interface. The top navigation bar includes the Kaggle logo, a search bar, and links for 'Iniciar sesión' and 'Registro'. The left sidebar contains a 'Crear' button and a list of navigation links: 'Inicio', 'Competiciones', 'Referencias', 'Arena de Juego', 'Centro de datos', and 'Más'. The main content area is titled 'Introducción a los agentes' and lists the authors: Alan Blount, Antonio Gulli, Shubham Saboo, Michael Zimmermann, and Vladimir Vuskovic. The 'Introducción' section contains two paragraphs of text about the evolution of AI and the role of agents. The bottom of the page shows a Windows taskbar with various application icons and system information.

Introducción a los agentes

Autores: Alan Blount, Antonio Gulli, Shubham Saboo, Michael Zimmermann y Vladimir Vuskovic

Introducción

La inteligencia artificial está cambiando. Durante años, el enfoque ha estado en modelos que destacan Tareas pasivas y discretas: responder a una pregunta, traducir texto o generar una imagen a partir de un prompt. Este paradigma, aunque poderoso, requiere una dirección humana constante en cada paso. Ahora estamos viendo un cambio de paradigma, pasando de una IA que solo predice o crea contenido a una nueva clase de software capaz de resolver problemas de forma autónoma y ejecutar tareas.

Esta nueva frontera está construida en torno a agentes de IA. Un agente no es simplemente un modelo de IA en un estado estático flujo de trabajo; Es una aplicación completa, haciendo planes y tomando acciones para alcanzar objetivos. Es combina la capacidad de razonar de un Modelo de Lenguaje (LM) con la capacidad práctica de actuar, permitiéndole manejar tareas complejas de varios pasos que un modelo por sí solo no puede. La capacidad crítica es que Los agentes pueden trabajar por su cuenta, averiguando los siguientes pasos necesarios para alcanzar un objetivo sin persona guiándoles en cada paso.

Lee el documento técnico a continuación

"Introducción a los agentes". Completa estos codelabs en Kaggle:

This screenshot is identical to the one above, showing the Kaggle website interface for the 'Introducción a los agentes' page. It includes the same navigation elements, title, authors, introductory text, and Windows taskbar.

Introducción a los agentes

Autores: Alan Blount, Antonio Gulli, Shubham Saboo, Michael Zimmermann y Vladimir Vuskovic

Introducción

La inteligencia artificial está cambiando. Durante años, el enfoque ha estado en modelos que destacan Tareas pasivas y discretas: responder a una pregunta, traducir texto o generar una imagen a partir de un prompt. Este paradigma, aunque poderoso, requiere una dirección humana constante en cada paso. Ahora estamos viendo un cambio de paradigma, pasando de una IA que solo predice o crea contenido a una nueva clase de software capaz de resolver problemas de forma autónoma y ejecutar tareas.

Esta nueva frontera está construida en torno a agentes de IA. Un agente no es simplemente un modelo de IA en un estado estático flujo de trabajo; Es una aplicación completa, haciendo planes y tomando acciones para alcanzar objetivos. Es combina la capacidad de razonar de un Modelo de Lenguaje (LM) con la capacidad práctica de actuar, permitiéndole manejar tareas complejas de varios pasos que un modelo por sí solo no puede. La capacidad crítica es que Los agentes pueden trabajar por su cuenta, averiguando los siguientes pasos necesarios para alcanzar un objetivo sin persona guiándoles en cada paso.

Lee el documento técnico a continuación

Construye tu primer agente usando Gemini y ADK

```
[32]: import os
      from kaggle_secrets import UserSecretsClient

      try:
          GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
          os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
          print("✅ Gemini API key setup complete.")
      except Exception as e:
          print(
              f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
          )

      ✅ Gemini API key setup complete.
```

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
: from google.adk.agents import Agent
  from google.adk.models.google_llm import Gemini
  from google.adk.runners import InMemoryRunner
  from google.adk.tools import google_search
  from google.genai import types

  print("✅ ADK components imported successfully.")

  ✅ ADK components imported successfully.
```

1.4: Helper functions

We'll define some helper functions. If you are running this outside the Kaggle environment, you don't need to do this.

+ Code

+ Markdown

```
4]: # Define helper functions that will be reused throughout the notebook

from IPython.core.display import display, HTML
from jupyter_server.serverapp import list_running_servers

# Gets the proxied URL in the Kaggle Notebooks environment
def get_adk_proxy_url():
    PROXY_HOST = "https://kkg-production.jupyter-proxy.kaggle.net"
    ADK_PORT = "8000"

    servers = list(list_running_servers())
    if not servers:
        raise Exception("No running Jupyter servers found.")

    baseURL = servers[0]["base_url"]

    try:
        path_parts = baseURL.split("/")
        kernel = path_parts[2]
        token = path_parts[3]
    except IndexError:
        raise Exception(f"Could not parse kernel/token from base URL: {baseURL}")

    url_prefix = f"/k/{kernel}/{token}/proxy/proxy/{ADK_PORT}"
    url = f"{PROXY_HOST}{url_prefix}"
```

2.2 Define your agent

Now, let's build our agent. We'll configure an `Agent` by setting its key properties, which tell it what to do and how to operate.

To learn more, check out the documentation related to [agents in ADK](#).

These are the main properties we'll set:

- **name** and **description**: A simple name and description to identify our agent.
- **model**: The specific LLM that will power the agent's reasoning. We'll use "gemini-2.5-flash-lite".
- **instruction**: The agent's guiding prompt. This tells the agent what its goal is and how to behave.
- **tools**: A list of `tools` that the agent can use. To start, we'll give it the `google_search` tool, which lets it find up-to-date information online.

```
37]: root_agent = Agent(
    name="root_agent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    description="A simple agent that can answer general questions.",
    instruction="You are a helpful assistant. Use Google Search for current info or if unsure.",
    tools=[google_search],
)

print("✅ Root Agent defined.")
```

✅ Root Agent defined.

2.3 Run your agent

Now it's time to bring your agent to life and send it a query. To do this, you need a `Runner`, which is the central component within ADK that acts as the orchestrator. It manages the conversation, sends our messages to the agent, and handles its responses.

a. Create an `InMemoryRunner` and tell it to use our `root_agent`:

```
: runner = InMemoryRunner(agent=root_agent)

print("✅ Runner created.")
```

✅ Runner created.

```
[39]: response = await runner.run_debug(
    "What is Agent Development Kit from Google? What languages is the SDK available in?"
)
```

```
### Created new session: debug_session_id
```

```
User > What is Agent Development Kit from Google? What languages is the SDK available in?
root_agent > The Google Agent Development Kit (ADK) is an open-source framework designed to simplify the development, deployment, and orchestration of AI agents and multi-agent systems. It aims to make building AI agents feel more like traditional software development, offering features like code-first development, modularity, and built-in debugging tools. The ADK is optimized for Google's Gemini models and Vertex AI but is model-agnostic and can be deployed in various environments.
```

```
The ADK is available in the following programming languages:
```

- * Python
- * Go
- * Java
- * TypeScript

You can see a summary of ADK and its available languages in the response.

```
[42]: response = await runner.run_debug("¿Cuál es la diferencia entre un LLM normal y un Agente de IA?")
```

```
### Continue session: debug_session_id
```

```
User > ¿Cuál es la diferencia entre un LLM normal y un Agente de IA?
root_agent > Un Modelo de Lenguaje Grande (LLM) es fundamentalmente un modelo de procesamiento de lenguaje, entrenado en vastas cantidades de texto para comprender y generar lenguaje humano. Su función principal es predecir la siguiente palabra en una secuencia, lo que le permite generar texto coherente, resumir documentos, traducir idiomas y responder preguntas. Los LLMs son la base de muchas aplicaciones de IA conversacionales, como los chatbots. Sin embargo, su capacidad de memoria se limita al contexto inmediato de la conversación y, por sí solos, son principalmente reactivos, es decir, responden a las entradas en lugar de tomar la iniciativa.
```

Un Agente de IA, por otro lado, es un sistema más complejo y dinámico diseñado para interactuar con su entorno y realizar acciones en función de objetivos específicos. Mientras que un LLM proporciona el "cerebro" o la capacidad de razonamiento, un agente de IA es el "cuerpo" que actúa en el mundo digital. Los agentes de IA pueden investigar, planificar, tomar decisiones, utilizar herramientas, aprender de sus interacciones y ajustar su comportamiento para alcanzar metas. A diferencia de los LLM, los agentes de IA están optimizados para completar tareas de varios pasos, incluso en presencia de incertidumbre y fallos.

En resumen:

- * **LLM:** Se enfoca en la comprensión y generación de lenguaje. Es una herramienta de procesamiento de texto.
- * **Agente de IA:** Utiliza un LLM como componente central, pero va más allá al tener la capacidad de interactuar con su entorno, tomar decisiones, usar herramientas y ejecutar acciones para lograr objetivos. Es un sistema autónomo diseñado para la acción y la consecución de metas.

Para tareas que requieren más que una simple respuesta textual, como la automatización de procesos empresariales o la ejecución de tareas complejas, un agente de IA es la solución más adecuada, ya que puede gestionar la toma de decisiones y la ejecución de acciones de manera más eficaz. La integración de ambas tecnologías puede ofrecer beneficios híbridos, aprovechando las fortalezas de cada una. Un Modelo de Lenguaje Grande (LLM) es fundamentalmente un modelo de procesamiento de lenguaje, entrenado en vastas cantidades de texto para comprender y generar lenguaje humano. Su función principal es predecir la siguiente palabra en una secuencia, lo que le permite generar texto coherente, resumir documentos, traducir idiomas y responder preguntas. Los LLMs son la base de muchas aplicaciones de IA conversacional, como los chatbots. Sin embargo, su capacidad de memoria se limita al contexto inmediato de la conversación y, por sí solos, son principalmente reactivos, es decir, responden a las entradas en lugar de tomar la iniciativa.

Un Agente de IA, por otro lado, es un sistema más complejo y dinámico diseñado para interactuar con su entorno y realizar acciones en función de objetivos específicos. Mientras que un LLM proporciona el "cerebro" o la capacidad de razonamiento, un agente de IA es el "cuerpo" que actúa

```
[43]: !adk create sample-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

```
Agent created in /kaggle/working/sample-agent:  
- .env  
- __init__.py  
- agent.py
```

+ Code

+ Markdown

Get your custom URL to access the ADK web UI in the Kaggle Notebooks environment:

```
[44]: url_prefix = get_adk_proxy_url()
```

⚠️ IMPORTANT: Action Required

The ADK web UI is **not running yet**. You must start it in the next cell.

1. **Run the next cell** (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, **return to this button** and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

Open ADK Web UI (after running cell below) ↗

```

/usr/local/lib/python3.11/dist-packages/google/ads/ccli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/ads/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [128]
INFO: Waiting for application startup.

```

```
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 35.191.17.53:0 - "GET / HTTP/1.1" 307 Temporary Redirect
INFO: 35.191.17.52:0 - "GET /dev-ui/ HTTP/1.1" 200 OK
INFO: 35.191.17.196:0 - "GET /dev-ui/polyfills-B6TN4ZQ6.js HTTP/1.1" 200 OK
INFO: 35.191.17.193:0 - "GET /dev-ui/main-0520H253.js HTTP/1.1" 200 OK
INFO: 35.191.17.150:0 - "GET /dev-ui/chunk-2KH2EVR6.js HTTP/1.1" 200 OK
INFO: 35.191.17.148:0 - "GET /dev-ui/styles-EVMP5V3U.css HTTP/1.1" 200 OK
INFO: 35.191.17.145:0 - "GET /dev-ui/assets/config/runtime-config.json HTTP/1.1" 200 OK
INFO: 35.191.17.144:0 - "GET /dev-ui/adk_favicon.svg HTTP/1.1" 200 OK
INFO: 35.191.17.192:0 - "GET /list-apps?relative_path=/ HTTP/1.1" 200 OK
INFO: 35.191.17.195:0 - "GET /dev-ui/assets/ADK-512-color.svg HTTP/1.1" 200 OK
INFO: google_adk_google_adk_cli.adk web server: New session created: 65eaab92-b9bd-4d3d-a9cc-0db966e185a8
INFO: 35.191.17.197:0 - "POST /apps/sample-agent/users/user/sessions HTTP/1.1" 200 OK
INFO: 35.191.17.194:0 - "GET /builder/app/sample-agent?ts=177703762413 HTTP/1.1" 200 OK
INFO: 35.191.17.198:0 - "GET /apps/sample-agent/users/user/sessions/65eaab92-b9bd-4d3d-a9cc-0db966e185a8 HTTP/1.1" 200 OK
INFO: 35.191.17.149:0 - "GET /apps/sample-agent/users/user/sessions HTTP/1.1" 200 OK
```

The screenshot shows a Kaggle Notebook environment. On the left, the 'Kit de Desarrollo de Agentes' sidebar is visible, featuring a chat window with the following content:

agente de muestra

Trace Eventos Estado Artefactos Sesiones

Invocaciones

¿Cuáles son las últimas noticias sobre computación cuántica

The main notebook area displays the following text:

Como modelo de lenguaje de IA, no tengo acceso a información en tiempo real ni a "noticias de hoy". Mi conocimiento se basa en los datos con los que fui entrenado, que no se actualizan constantemente. Sin embargo, puedo ofrecerte algunas áreas generales donde se está produciendo mucha actividad e innovación en computación cuántica:

- Avances en hardware:** Empresas y laboratorios de investigación están constantemente anunciando mejoras en la calidad y cantidad de qubits que pueden controlar, así como en la reducción de errores. Esto incluye diferentes enfoques como qubits superconductores, trampas de iones, fotónicos, etc.
- Desarrollo de algoritmos:** Se sigue investigando en la creación de algoritmos cuánticos que puedan resolver problemas específicos de manera más eficiente que las computadoras clásicas. Esto incluye áreas como la química cuántica, la optimización, el descubrimiento de fármacos y la ciencia de materiales.
- Software y herramientas:** Se están desarrollando plataformas de software, lenguajes de programación y herramientas de simulación para facilitar el acceso y uso de las computadoras cuánticas, incluso para aquellos que no son expertos en física cuántica.
- Aplicaciones prácticas:** Aunque todavía estamos en etapas tempranas, se están explorando activamente las aplicaciones potenciales en diversas industrias, como la financiera, la farmacéutica, la logística y la inteligencia artificial.
- Colaboraciones y financiación:** Hay una gran inversión y colaboración entre empresas, gobiernos y universidades para impulsar el campo de la computación cuántica.

Para obtener las noticias más recientes y específicas de hoy, te recomiendo consultar fuentes confiables como:

- Sitios web de noticias de tecnología y ciencia:** Busca secciones dedicadas a la computación cuántica en sitios como Nature, Science, MIT Technology Review, IEEE Spectrum, Ars Technica, etc.
- Boletines y fuentes de empresas líderes:** IBM Quantum, Google AI Quantum, Microsoft Azure Quantum, etc.

At the bottom of the sidebar, there is a chat input field with the placeholder text 'Escribe un mensaje...' and icons for sending, adding attachments, voice recording, and video recording.

Construye tus primeros sistemas multiagente usando ADK.

2.1 Example: Research & Summarization System

Let's build a system with two specialized agents:

1. **Research Agent** - Searches for information using Google Search
2. **Summarizer Agent** - Creates concise summaries from research findings

```
# Research Agent: Its job is to use the google_search tool and present findings.
research_agent = Agent(
    name="ResearchAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""You are a specialized research agent. Your only job is to use the
google_search tool to find 2-3 pieces of relevant information on the given topic and present the findings with citations.""",
    tools=[google_search],
    output_key="research_findings", # The result of this agent will be stored in the session state with this key.
)

print("✅ research_agent created.")
```

✅ research_agent created.

+ Code + Markdown

```
# Summarizer Agent: Its job is to summarize the text it receives.
summarizer_agent = Agent(
    name="SummarizerAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # The instruction is modified to request a bulleted list for a clear output format.
    instruction="""Read the provided research findings: {research_findings}
Create a concise summary as a bulleted list with 3-5 key points.""",
    output_key="final_summary",
)

print("✅ summarizer_agent created.")
```

✅ summarizer_agent created.

+ Code + Markdown

Refer to the ADK documentation for more information on guiding agents with clear and specific instructions.

Then we bring the agents together under a root agent, or coordinator:

```
# Root Coordinator: Orchestrates the workflow by calling the sub-agents as tools.
root_agent = Agent(
    name="ResearchCoordinator",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # This instruction tells the root agent HOW to use its tools (which are the other agents).
    instruction="""You are a research coordinator. Your goal is to answer the user's query by orchestrating a workflow.
1. First, you MUST call the 'ResearchAgent' tool to find relevant information on the topic provided by the user.
2. Next, after receiving the research findings, you MUST call the 'SummarizerAgent' tool to create a concise summary.
3. Finally, present the final summary clearly to the user as your response.""",
    # We wrap the sub-agents in 'AgentTool' to make them callable tools for the root agent.
    tools=[AgentTool(research_agent), AgentTool(summarizer_agent)],
)

print("✅ root_agent created.")
```

✅ root_agent created.

+ Code + Markdown

Here we're using `AgentTool` to wrap the sub-agents to make them callable tools for the root agent. We'll explore `AgentTool` in-detail on Day 2.

Let's run the agent and ask it about a topic:

⤵

↑ ↓ 📄

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "What are the latest advancements in quantum computing and what do they mean for AI?"
)

### Created new session: debug_session_id

User > What are the latest advancements in quantum computing and what do they mean for AI?
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
ResearchCoordinator > The latest advancements in quantum computing are poised to significantly boost AI capabilities by offering exponential increases in computational power and speed. This allows for much faster processing of vast datasets, leading to more efficient AI models and quicker training times. Quantum computing can also enable AI to tackle complex problems currently beyond the reach of classical computers, with applications in fields like drug discovery and materials science.

Key breakthroughs include advancements in quantum error correction, which are essential for building stable quantum computers, and the development of hybrid quantum-classical approaches that combine the strengths of both technologies. These developments promise to enhance AI's predictive capabilities, improve data processing, and accelerate innovation across various industries.
```

+ Code + Markdown

You've just built your first multi-agent system! You used a single "coordinator" agent to manage the workflow, which is a powerful and flexible pattern.

!! However, **relying on an LLM's instructions to control the order can sometimes be unpredictable**. Next, we'll explore a different pattern that gives you guaranteed, step-by-step execution.



3.1 Example: Blog Post Creation with Sequential Agents

Let's build a system with three specialized agents:

1. **Outline Agent** - Creates a blog outline for a given topic
2. **Writer Agent** - Writes a blog post
3. **Editor Agent** - Edits a blog post draft for clarity and structure

```
# Outline Agent: Creates the initial blog post outline.
outline_agent = Agent(
    name="OutlineAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Create a blog outline for the given topic with:
1. A catchy headline
2. An introduction hook
3. 3-5 main sections with 2-3 bullet points for each
4. A concluding thought""",
    output_key="blog_outline", # The result of this agent will be stored in the session state with this key
)

print("✅ outline_agent created.")
```

✅ outline_agent created.

+ Code + Markdown

```
# Writer Agent: Writes the full blog post based on the outline from the previous agent.
writer_agent = Agent(
    name="WriterAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # The '{blog_outline}' placeholder automatically injects the state value from the previous agent's output
    instruction="""Following this outline strictly: {blog_outline}
Write a brief, 200 to 300-word blog post with an engaging and informative tone.""",
    output_key="blog_draft", # The result of this agent will be stored with this key.
)

print("✅ writer_agent created.")
```

Execute cell. Cell execution started at 1:20am

✅ writer_agent created.

+ Code + Markdown

```
# Editor Agent: Edits and polishes the draft from the writer agent.
Execute cell. Cell execution started at 1:20am
name="EditorAgent",
model=Gemini(
    model="gemini-2.5-flash-lite",
    retry_options=retry_config
),
# This agent receives the `{blog_draft}` from the writer agent's output.
instruction="""Edit this draft: {blog_draft}
Your task is to polish the text by fixing any grammatical errors, improving the flow and sentence s
output_key="final_blog", # This is the final output of the entire pipeline.
)

print("✅ editor_agent created.")
```

✅ editor_agent created.

+ Code + Markdown

Then we bring the agents together under a sequential agent, which runs the agents in the order that they are listed:

```
root_agent = SequentialAgent(
    name="BlogPipeline",
    sub_agents=[outline_agent, writer_agent, editor_agent],
)

print("✅ Sequential Agent created.")
```

✅ Sequential Agent created.

+ Code + Markdown

Let's run the agent and give it a topic to write a blog post about:

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "Write a blog post about the benefits of multi-agent systems for software developers"
)
```

```
### Created new session: debug_session_id

User > Write a blog post about the benefits of multi-agent systems for software developers
OutlineAgent > Here is a blog outline about the benefits of multi-agent systems for software developers:

## **Unleash Your Code's Potential: Why Multi-Agent Systems Are Your Next Big Breakthrough**

Are you a software developer constantly seeking ways to build more robust, intelligent, and adaptable application
s? Imagine a world where your software components don't just execute instructions but actively collaborate, lear
n, and solve problems with unprecedented efficiency. Welcome to the transformative power of Multi-Agent Systems
(MAS).

### **Enhanced Problem Solving and Complexity Management**
```

+ Code

+ Markdown

↑ ↓ 🗑

▶

```
# Tech Researcher: Focuses on AI and ML trends.
tech_researcher = Agent(
    name="TechResearcher",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Research the latest AI/ML trends. Include 3 key developments,
the main companies involved, and the potential impact. Keep the report very concise (100 words).""",
    tools=[google_search],
    output_key="tech_research", # The result of this agent will be stored in the session state with this key.
)

print("✅ tech_researcher created.")
```

✅ tech_researcher created.

+ Code

+ Markdown

+ Code

+ Markdown

↑ ↓ 🗑

▶

```
# Health Researcher: Focuses on medical breakthroughs.
health_researcher = Agent(
    name="HealthResearcher",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Research recent medical breakthroughs. Include 3 significant advances,
their practical applications, and estimated timelines. Keep the report concise (100 words).""",
    tools=[google_search],
    output_key="health_research", # The result will be stored with this key.
)

print("✅ health_researcher created.")
```

✅ health_researcher created.

+ Code

+ Markdown

+ Code

+ Markdown

↑ ↓ 🗑

▶

Execute cell. Cell executed in 0.005s at 1:26am

```
# Finance Researcher: Focuses on fintech trends.
name="FinanceResearcher",
model=Gemini(
    model="gemini-2.5-flash-lite",
    retry_options=retry_config
),
instruction="""Research current fintech trends. Include 3 key trends,
their market implications, and the future outlook. Keep the report concise (100 words).""",
tools=[google_search],
output_key="finance_research", # The result will be stored with this key.
)

print("✅ finance_researcher created.")
```

✅ finance_researcher created.

+ Code

+ Markdown

```
# The AggregatorAgent runs *after* the parallel step to synthesize the results.
aggregator_agent = Agent(
    name="AggregatorAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # It uses placeholders to inject the outputs from the parallel agents, which are now in the session s
    instruction="""Combine these three research findings into a single executive summary:

    **Technology Trends:**
    {tech_research}

    **Health Breakthroughs:**
    {health_research}

    **Finance Innovations:**
    {finance_research}

    Your summary should highlight common themes, surprising connections, and the most important key take
    output_key="executive_summary", # This will be the final output of the entire system.
)

print("✅ aggregator_agent created.")
```

✅ aggregator_agent created.

+ Code + Markdown

```
# The ParallelAgent runs all its sub-agents simultaneously.
parallel_research_team = ParallelAgent(
    name="ParallelResearchTeam",
    sub_agents=[tech_researcher, health_researcher, finance_researcher],
).

# This SequentialAgent defines the high-level workflow: run the parallel team first, then run the aggregator
root_agent = SequentialAgent(
    name="ResearchSystem",
    sub_agents=[parallel_research_team, aggregator_agent],
)

print("✅ Parallel and Sequential Agents created.")
```

✅ Parallel and Sequential Agents created.

+ Code + Markdown

Let's run the agent and give it a prompt to research the given topics:

Day 1b - Agent Architectures

Archivo Edición Vista Corre Escenarios Complementos Ayuda

Ejecutar todo Código Sesión de borrador (33m)

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "Run the daily executive briefing on Tech, Health, and Finance"
)

### Created new session: debug_session_id

User > Run the daily executive briefing on Tech, Health, and Finance
HealthResearcher > Here are three recent medical breakthroughs:

1. **AI-Powered Drug Discovery:** Artificial intelligence is revolutionizing drug development, drastically reducing the time and cost. AI can now identify potential drug candidates and predict their efficacy with 100% accuracy in as little as 30 days, a process that previously took about ten years and billions of dollars. This advance promises to accelerate treatments for various diseases.

2. **Gene editing with CRISPR:** CRISPR gene-editing technology allows for precise modification of DNA sequence. Its applications range from potentially curing hereditary diseases to developing disease-resistant crops. This technology is rapidly being adopted globally.

3. **Advanced Stroke Treatment:** Neurovascular stent retrievers offer a new, minimally invasive approach to removing blood clots causing strokes. This technology can significantly improve recovery times and regain normal function, with optimal results achieved by removing clots within 2-7 hours of a stroke.

TechResearcher > Here's a concise executive briefing on the latest AI/ML trends:

**Key Developments:**

1. **Agentic AI & Autonomous Systems:** AI is moving beyond assistance to independent action, capable of planning, decision-making, and task execution without constant human input. This is transforming core operations in finance and beyond.

2. **Multimodal AI:** Systems are increasingly adept at processing and generating diverse data types (text, images, audio, video) simultaneously, leading to more context-aware and adaptable AI behavior across industries.

3. **AI as Foundational Infrastructure:** AI is becoming deeply integrated into enterprise architecture and software development lifecycles, shifting from a feature to a core component enabling new levels of efficiency and innovation.
```

Cuaderno

Entrada

+ Añadir entrada Subida

Sin entrada adjunta

Adjuntar un conjunto de datos, modelo o competencia de Kaggle

Salida (84KiB / 19,5GiB)

/kaggle/trabajando

Índice

Sistemas Multiagente y Patrones de Flujo ...

¡ Por favor, lee

```
# This agent runs ONCE at the beginning to create the first draft.
initial_writer_agent = Agent(
    name="InitialWriterAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="Based on the user's prompt, write the first draft of a short story (around 100-150 words). Output only the story text, with no introduction or explanation.",
    output_key="current_story", # Stores the first draft in the state.
)

print("✅ initial_writer_agent created.")
```

✅ initial_writer_agent created.

+ Code + Markdown

```
# This agent's only job is to provide feedback or the approval signal. It has no tools.
name="CriticAgent",
model=Gemini(
    model="gemini-2.5-flash-lite",
    retry_options=retry_config
),
instruction="""You are a constructive story critic. Review the story provided below.
Story: {current_story}

Evaluate the story's plot, characters, and pacing.
- If the story is well-written and complete, you MUST respond with the exact phrase: "APPROVED"
- Otherwise, provide 2-3 specific, actionable suggestions for improvement."""
output_key="critique", # Stores the feedback in the state.
)

print("✅ critic_agent created.")
```

✅ critic_agent created.

+ Code + Markdown

Now, we need a way for the loop to actually stop based on the critic's feedback. The `LoopAgent` itself doesn't automatically know that "APPROVED" means "stop."

We need an agent to give it an explicit signal to terminate the loop.

Let's define the `exit_loop` function.

```
! This is the function that the RefinerAgent will call to exit the loop.
"""Call this function ONLY when the critique is 'APPROVED', indicating the story is finished and no more refinements are needed."""
return {"status": "approved", "message": "Story approved. Exiting refinement loop."}

print("✅ exit_loop function created.")
```

✅ exit_loop function created.

+ Code + Markdown

To let an agent call this Python function, we wrap it in a `FunctionTool`. Then, we create a `RefinerAgent` that has this tool.

👉 **Notice its instructions:** this agent is the "brain" of the loop. It reads the `{critique}` from the `CriticAgent` and decides whether

```
# This agent refines the story based on critique OR calls the exit_loop function.
refiner_agent = Agent(
    name="RefinerAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""You are a story refiner. You have a story draft and critique.

    Story Draft: {current_story}
    Critique: {critique}

    Your task is to analyze the critique.
    - IF the critique is EXACTLY "APPROVED", you MUST call the `exit_loop` function and nothing else.
    - OTHERWISE, rewrite the story draft to fully incorporate the feedback from the critique.""",
    output_key="current_story", # It overwrites the story with the new, refined version.
    tools=[
        FunctionTool(exit_loop)
    ], # The tool is now correctly initialized with the function reference.
)

print("✅ refiner_agent created.")
```

✅ refiner_agent created.

+ Code + Markdown

Then we bring the agents together under a loop agent, which is itself nested inside of a sequential agent.

```
# The LoopAgent contains the agents that will run repeatedly: Critic -> Refiner.
story_refinement_loop = LoopAgent(
    name="StoryRefinementLoop",
    sub_agents=[critic_agent, refiner_agent],
    max_iterations=2, # Prevents infinite loops
)

# The root agent is a SequentialAgent that defines the overall workflow: Initial Write -> Refinement Loop
root_agent = SequentialAgent(
    name="StoryPipeline",
    sub_agents=[initial_writer_agent, story_refinement_loop],
)

print("✅ Loop and Sequential Agents created.")
```

✅ Loop and Sequential Agents created.

+ Code + Markdown

Let's run the agent and give it a topic to write a short story about:

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "Write a short story about a lighthouse keeper who discovers a mysterious, glowing map"
)

### Created new session: debug_session_id

User > Write a short story about a lighthouse keeper who discovers a mysterious, glowing map
```

+ Code + Markdown

You've now implemented a loop agent, creating a sophisticated system that can iteratively review and improve its own output. This is a key pattern for ensuring high-quality results.

You now have a complete toolkit of workflow patterns. Let's put it all together and review how to choose the right one for your use case.

Day 1b - Agent Architectures

Section 6: Summary - Choosing the Right Pattern

Decision Tree: Which Workflow Pattern?

```
graph TD
    Root{What kind of workflow do you need?} --> Fixed["Fixed Pipeline  
(A -> B -> C)"]
    Root --> Concurrent["Concurrent Tasks  
(Run A, B, C all at once)"]
    Root --> Iterative["Iterative Refinement  
(A ≈ B)"]
    Root --> Dynamic["Dynamic Decisions  
(Let the LLM decide what to do)"]
    Fixed --> Sequential["Use SequentialAgent"]
    Concurrent --> Parallel["Use ParallelAgent"]
    Iterative --> Loop["Use LoopAgent"]
    Dynamic --> Orchestrator["Use LLM Orchestrator  
(Agent with other agents as tools)"]
```

Salida (84KiB / 19,5GiB)

/kaggle/trabajando

Índice

Sistemas Multiagente y Patrones de Flujo ...

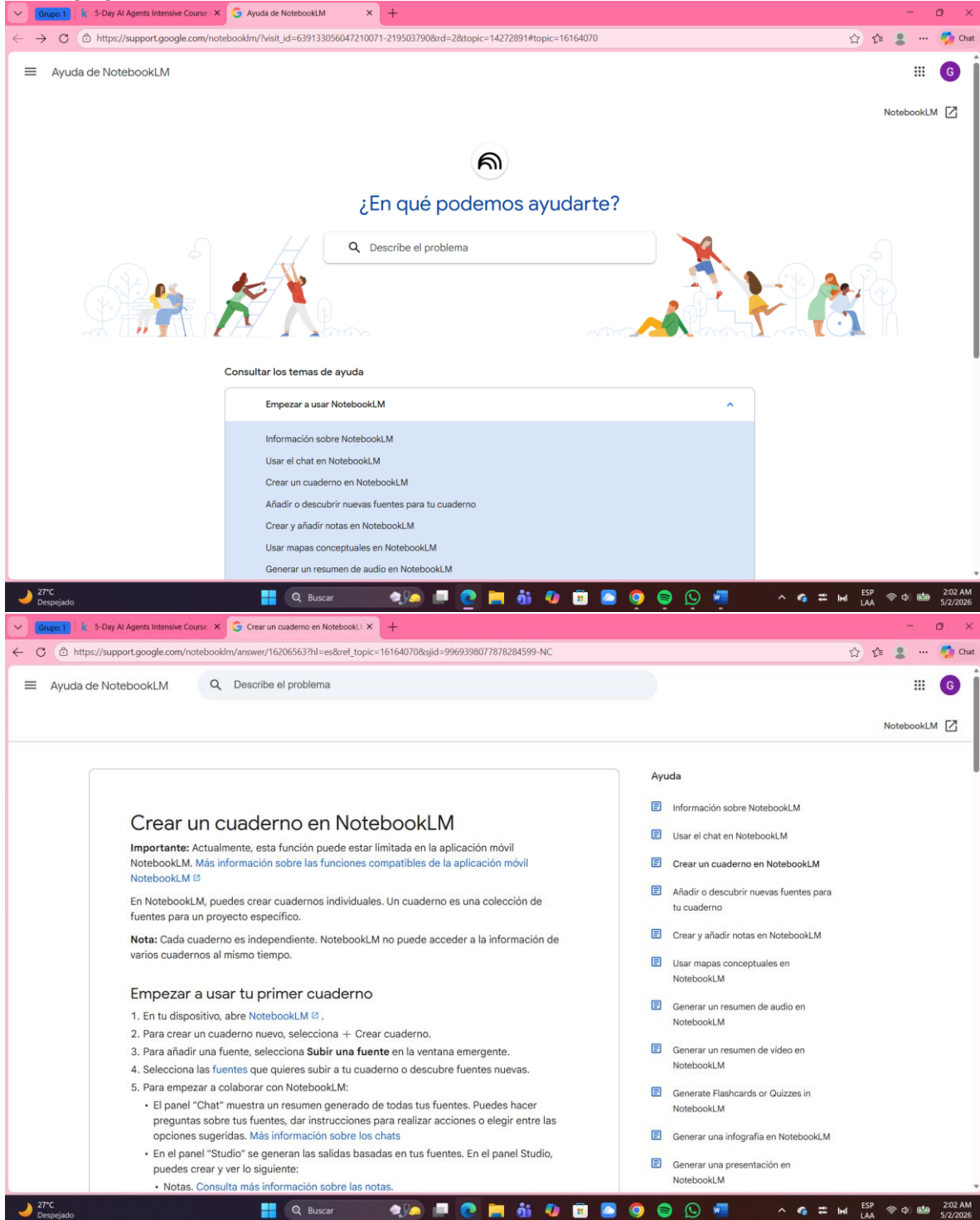
¡ Por favor, lee

Asegúrate de verificar tu cuenta de Kaggle por teléfono antes de empezar, es necesario para los codelabs. También tenemos una guía de solución de problemas para los codelabs. □

The screenshot shows the Kaggle account verification page. The browser tabs include 'Grupo 1', '5-Day AI Agents Intensive Course', 'Escenarios | Kaggle', 'Day 1b - Agent Architectures', 'Kaggle Notebook Editor', and 'Day 1b - Agent Architectures'. The page title is 'Escenarios' and the subtitle is 'Control sobre tu cuenta de Kaggle y todas las comunicaciones'. The left sidebar contains navigation links: 'Crear', 'Inicio', 'Competiciones', 'Referencias', 'Arena de Juego', 'Centro de datos', 'Más', 'Tu obra', 'Visto', 'Día 1b - Arquitectura...', 'Día 1a - De la Consig...', 'Día 1a - De la Consig...', 'Editado', 'Día 1b - Arquitectura...', 'Día 1a - De la Consig...', 'Día 1a - De la Consig...', and 'Ver eventos activos'. The main content area shows account details: 'Tu dirección de correo electrónico' (guerrero vasquez fernandaximena@gmail.com), 'Tu nombre de usuario' (Fernandaximena), and 'Tu número de cuenta' (33979762). There are buttons for 'Cambiar de correo electrónico', 'Verificación telefónica' (Verificado), 'Verificación de identidad' (No has verificado tu identidad usando Persona), and 'Tema' (Elige tu tema de interfaz de Kaggle a continuación).

The screenshot shows the Kaggle troubleshooting and FAQs page. The browser tabs include 'Grupo 1', '5-Day AI Agents Intensive Course', and 'Day 0 - Troubleshooting and FAQs'. The page title is 'Día 0 - Resolución de problemas y preguntas frecuentes'. The left sidebar contains navigation links: 'Crear', 'Inicio', 'Competiciones', 'Referencias', 'Arena de Juego', 'Centro de datos', 'Más', 'Tu obra', 'Visto', 'Día 0 - Resolución de...', 'Día 1b - Arquitectura...', 'Día 1a - De la Consig...', 'Día 1a - De la Consig...', 'Editado', 'Día 1b - Arquitectura...', 'Día 1a - De la Consig...', 'Día 1a - De la Consig...', and 'Ver eventos activos'. The main content area shows the notebook 'Troubleshooting errors and FAQs' with a duration of 9s and a language of Python. The text states: 'This notebook covers some common questions and problems that people may encounter while following the material in the Kaggle 5-Day AI Agents Intensive Course with Google. It is not intended to be part of the coursework but please check here for your problem before asking in the Discord or posting elsewhere.' The 'General Questions' section includes links to the course and the Discord server. The 'Índice' section lists various topics: 'Errores de resolución de problemas', 'Preguntas generales', 'Preguntas frecuentes sobre configuración', 'Preguntas frecuentes sobre los cursos', 'Preguntas frecuentes sobre Google', 'Preguntas frecuentes sobre los cursos', and 'Errores'.

¿Quieres tener una conversación interactiva? Prueba a añadir los whitepapers a NotebookLM.



The screenshot shows the Google NotebookLM help page. At the top, there's a search bar with the placeholder text "Describe el problema". Below the search bar, there's a section titled "Consultar los temas de ayuda" (Consult the help topics). This section contains a list of topics to explore, including "Empezar a usar NotebookLM" (Get started with NotebookLM), "Información sobre NotebookLM" (Information about NotebookLM), "Usar el chat en NotebookLM" (Use the chat in NotebookLM), "Crear un cuaderno en NotebookLM" (Create a notebook in NotebookLM), "Añadir o descubrir nuevas fuentes para tu cuaderno" (Add or discover new sources for your notebook), "Crear y añadir notas en NotebookLM" (Create and add notes in NotebookLM), "Usar mapas conceptuales en NotebookLM" (Use concept maps in NotebookLM), and "Generar un resumen de audio en NotebookLM" (Generate an audio summary in NotebookLM).

Below the list of topics, there's a section titled "Crear un cuaderno en NotebookLM" (Create a notebook in NotebookLM). This section includes an important note: "Importante: Actualmente, esta función puede estar limitada en la aplicación móvil NotebookLM. Más información sobre las funciones compatibles de la aplicación móvil NotebookLM" (Important: Currently, this feature may be limited in the mobile NotebookLM app. More information about compatible features of the mobile NotebookLM app). It also includes a note: "Nota: Cada cuaderno es independiente. NotebookLM no puede acceder a la información de varios cuadernos al mismo tiempo." (Note: Each notebook is independent. NotebookLM cannot access information from multiple notebooks at the same time).

The section "Empezar a usar tu primer cuaderno" (Get started with your first notebook) provides a list of steps to follow:

1. En tu dispositivo, abre [NotebookLM](#).
2. Para crear un cuaderno nuevo, selecciona [+ Crear cuaderno](#).
3. Para añadir una fuente, selecciona [Subir una fuente](#) en la ventana emergente.
4. Selecciona las [fuentes](#) que quieres subir a tu cuaderno o descubre fuentes nuevas.
5. Para empezar a colaborar con NotebookLM:
 - El panel "Chat" muestra un resumen generado de todas tus fuentes. Puedes hacer preguntas sobre tus fuentes, dar instrucciones para realizar acciones o elegir entre las opciones sugeridas. [Más información sobre los chats](#)
 - En el panel "Studio" se generan las salidas basadas en tus fuentes. En el panel Studio, puedes crear y ver lo siguiente:
 - Notas. [Consulta más información sobre las notas.](#)

On the right side of the page, there's a sidebar titled "Ayuda" (Help) with a list of topics to explore, including "Información sobre NotebookLM", "Usar el chat en NotebookLM", "Crear un cuaderno en NotebookLM", "Añadir o descubrir nuevas fuentes para tu cuaderno", "Crear y añadir notas en NotebookLM", "Usar mapas conceptuales en NotebookLM", "Generar un resumen de audio en NotebookLM", "Generar un resumen de video en NotebookLM", "Generate Flashcards or Quizzes in NotebookLM", "Generar una infografía en NotebookLM", and "Generar una presentación en NotebookLM".

¿Quieres tener una conversación interactiva? Prueba a añadir los whitepapers a NotebookLM.

The Architecture of Autonomous Python AI Agents

1 fuente · 2 may 2026

Esta fuente describe la evolución de los **agentes de inteligencia artificial** en Python, los cuales han pasado de ser simples scripts a sistemas con capacidad de **razonamiento y autonomía**. El texto detalla una estructura compuesta por cinco pilares fundamentales: un **modelo de lenguaje** que actúa como cerebro, herramientas externas, memoria de contexto, orquestación de tareas y un sistema de instrucciones precisas. Se destaca el uso del **patrón ReAct**, un ciclo donde el agente piensa, actúa y observa para cumplir objetivos complejos. Además, se explica el flujo de **planificación y ejecución**, donde el sistema desglosa una meta general en tareas técnicas procesables. En conjunto, el documento ofrece una visión técnica sobre cómo estos agentes utilizan **funciones de Python** y modelos avanzados para interactuar con su entorno de manera inteligente.

Guardar en las notas · Comienza a escribir...

1 fuente

Los datos de salida de Studio se guardarán aquí. Luego de agregar fuentes, haz clic para agregar un resumen de audio, una guía de estudio, un mapa mental y más.

Agregar nota

The Architecture of Autonomous Python AI Agents

1 fuente · 2 may 2026

Las fuentes describen la evolución de los **agentes de inteligencia artificial** en Python, los cuales han pasado de ser simples scripts a sistemas con capacidad de **razonamiento y planificación**. Un agente moderno se fundamenta en un **modelo de lenguaje (LLM)** que actúa como cerebro, procesando información y tomando decisiones de manera autónoma. Para interactuar con su entorno, estos sistemas utilizan **herramientas externas**, una memoria que retiene el contexto y un **bucle de orquestación** basado en el patrón Pensar-Actuar-Observar. Además, se destaca el uso de una arquitectura de dos etapas donde un **planificador** desglosa los objetivos en tareas concretas que un **ejecutor** lleva a cabo mediante código. En conjunto, estos elementos permiten que la IA no solo responda preguntas, sino que cumpla objetivos complejos mediante la integración de herramientas externas y funciones.

Comienza a escribir...

1 fuente

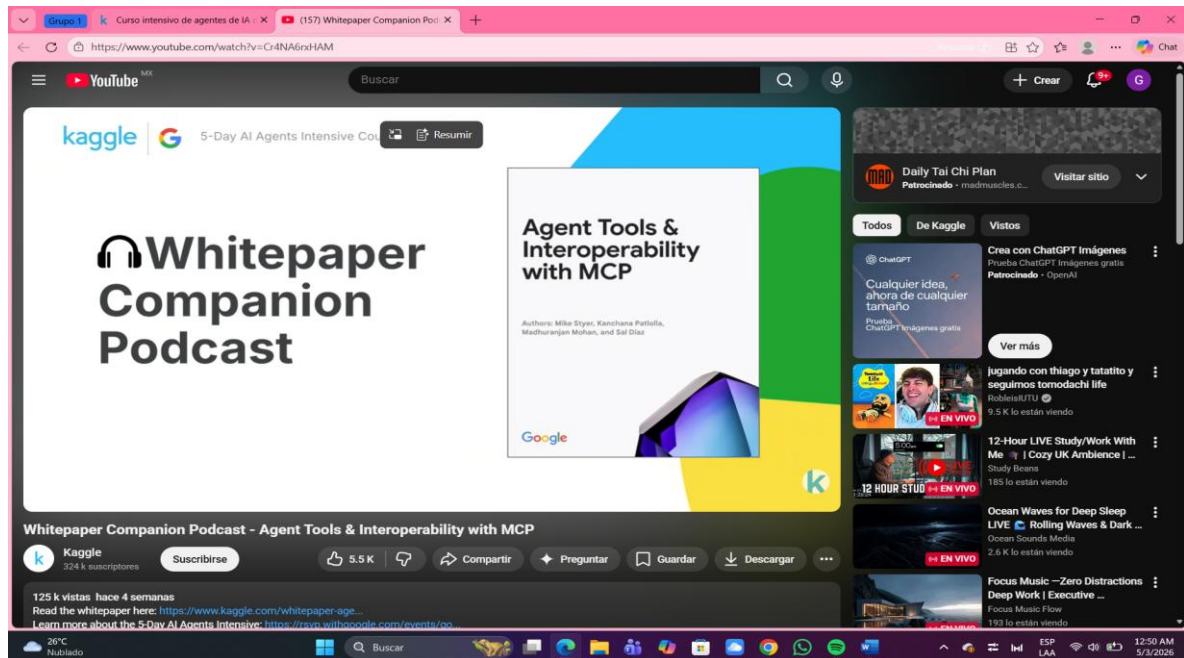
Los datos de salida de Studio se guardarán aquí. Luego de agregar fuentes, haz clic para agregar un resumen de audio, una guía de estudio, un mapa mental y más.

Agregar nota

The screenshot shows the Kaggle website interface. On the left is a sidebar with navigation options: 'Inicio', 'Competiciones', 'Referencias', 'Arena de Juego', 'Centro de datos', 'Más', 'Tu obra', 'Visto', and a list of recent projects. The main content area features a search bar at the top. Below it is a video player titled 'Día 1 (Introducción a los agentes)'. The text on the page welcomes users to Day 1 and explains the document's purpose: to present a taxonomy of agent capabilities and discuss the need for a discipline of Agent Operations for reliability and governance. It also mentions that users will build their first agent using the Agent Development Kit (ADK) and Google Gemini. A list of tasks for the day is provided, including listening to a podcast, reading a whitepaper, and building an agent. The page concludes with an optional section about watching a live recording of the first retransmission.

Día 2 (Herramientas de Agente e Interoperabilidad con el Protocolo de Contexto de Modelo (MCP))

Escucha el episodio resumen del podcast de esta unidad.



Para complementar el pódcast, lee el whitepaper "Agent Tools & Interoperability with Model Context Protocol (MCP)".



Explora nuevas formas de añadir herramientas que amplíen lo que tus agentes pueden hacer.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(
        f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets"
    )
```

✅ Setup and authentication complete.

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.genai import types

from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import google_search, AgentTool, ToolContext
from google.adk.code_executors import BuiltInCodeExecutor

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

+ Code + Markdown

1.4: Helper functions

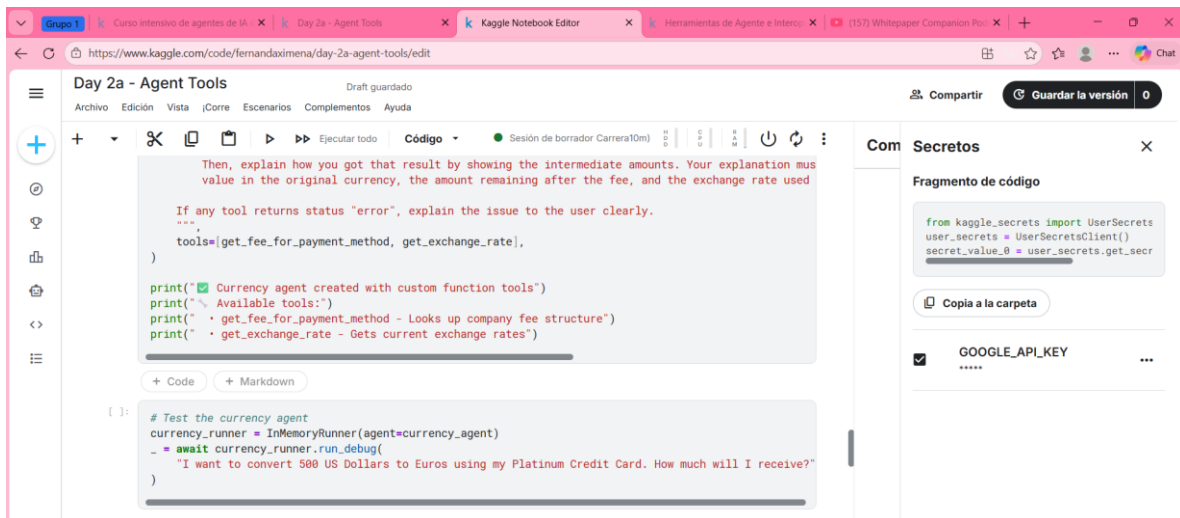
Helper function that prints the generated Python code and results from the code execution tool:

```
def show_python_code_and_result(response):
    for i in range(len(response)):
        # Check if the response contains a valid function call result from the code executor
        if (
            (response[i].content.parts
             and (response[i].content.parts[0]
                 and (response[i].content.parts[0].function_response
                     and (response[i].content.parts[0].function_response.response)
                )
            ):
            response_code = response[i].content.parts[0].function_response.response
            if "result" in response_code and response_code["result"] != "":
                if "tool_code" in response_code["result"]:
                    print(
                        "Generated Python Code >> ",
                        response_code["result"].replace("tool_code", ""),
                    )
                else:
                    print("Generated Python Response >> ", response_code["result"])

    print("✅ Helper functions defined.")
```

✅ Helper functions defined.

+ Code + Markdown




```
# Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
- = await currency_runner.run_debug(
    "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)

### Created new session: debug_session_id

User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call', 'function_call'],
returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
currency_agent > You will receive 450.8 Euros.

Here's how that's calculated:

1. **Fee deduction:** A 2% fee is charged for using a platinum credit card. This amounts to 10 USD (500 USD * 0.02).
2. **Amount after fee:** After deducting the fee, you have 490 USD remaining (500 USD - 10 USD).
3. **Currency conversion:** Using the exchange rate of 0.93, your 490 USD is converted to 455.7 Euros (490 USD * 0.93).
```

Excellent! Our agent now uses custom business logic with structured responses.

Grupo 1

Curso intensivo de agentes de IA

Day 2a - Agent Tools

Kaggle Notebook Editor

Herramientas de Agente e Interop

(157) Whitepaper Co

https://www.kaggle.com/code/fernandaximena/day-2a-agent-tools/edit

Day 2a - Agent Tools

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ - ✂ 📄 📋 ▶ ▶▶ Ejecutar todo Código Sesión de borrador Carrera20m

+

🔍

🔒

🔗

🔧

🔌

🔄

⋮

for any currency conversion request

1. Get Transaction Fee: Use the get_fee_for_payment_method() tool to determine the transaction fee.
2. Get Exchange Rate: Use the get_exchange_rate() tool to get the currency conversion rate.
3. Error Check: After each tool call, you must check the "status" field in the response. If the stat
4. Calculate Final Amount (CRITICAL): You are strictly prohibited from performing any arithmetic cal
code will use the fee information from step 1 and the exchange rate from step 2.
5. Provide Detailed Breakdown: In your summary, you must:
* State the final converted amount.
* Explain how the result was calculated, including:
* The fee percentage and the fee amount in the original currency.
* The amount remaining after deducting the fee.
* The exchange rate applied.
"""
tools=[
 get_fee_for_payment_method,
 get_exchange_rate,
 AgentTool(agent=calculation_agent), # Using another agent as a tool!
],
)

print("✅ Enhanced currency agent created")
print("🔗 New capability: Delegates calculations to specialist agent")
print("🔍 Tool types used:")
print("• Function Tools (fees, rates)")
print("• Agent Tool (calculation specialist)")

✅ Enhanced currency agent created
🔗 New capability: Delegates calculations to specialist agent
🔍 Tool types used:
• Function Tools (fees, rates)
• Agent Tool (calculation specialist)

+ Code + Markdown

Com

Sec

Fraç

fr
us
se

📄

✅

+

Day 2a - Agent Tools

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ Ejecutar todo Código Sesión de borrador Carrera29m

```
[21]: # Define a runner
enhanced_runner = InMemoryRunner(agent=enhanced_currency_agent)

[18]: # Test the enhanced agent
response = await enhanced_runner.run_debug(
    "Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation."
)

### Created new session: debug_session_id

User > Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['executable_code', 'code_execution_result'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
enhanced_currency_agent > Sorry for the trouble. It seems that there was an issue with the calculation. I will perform this calculation again.

The transaction fee for a bank transfer is 1%, and the exchange rate from USD to INR is 83.58.
```

Compartir Guardar la versión 0

Secretos

Fragmento de código

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copia a la carpeta

GOOGLE_API_KEY

+ Añadir secreto Fragmento de copia

Day 2a - Agent Tools

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ Ejecutar todo Código Sesión de borrador Carrera29m

```
[22]: show_python_code_and_result(response)

Generated Python Response >> '''python
amount_usd = 1250
fee_percentage = 0.01
exchange_rate = 83.58
fee_amount = amount_usd * fee_percentage
amount_after_fee = amount_usd - fee_amount
converted_amount = amount_after_fee * exchange_rate
print(f"fee percentage: {fee_percentage}")
print(f"fee amount in USD: {fee_amount}")
print(f"Amount after fee in USD: {amount_after_fee}")
print(f"Exchange rate: {exchange_rate}")
print(f"final converted amount in INR: {converted_amount}")
'''

Generated Python Response >> '''python
amount_usd = 1250
fee_percentage = 0.01
exchange_rate = 83.58
fee_amount = amount_usd * fee_percentage
amount_after_fee = amount_usd - fee_amount
converted_amount = amount_after_fee * exchange_rate
print(f"fee percentage: {fee_percentage}")
print(f"fee amount in USD: {fee_amount}")
print(f"Amount after fee in USD: {amount_after_fee}")
print(f"Exchange rate: {exchange_rate}")
print(f"final converted amount in INR: {converted_amount}")
'''
```

3.3: Agent Tools vs Sub-Agents: What's the Difference?

This is a common question! Both involve using multiple agents, but they work very differently.

Compartir Guardar la versión 0

Secretos

Fragmento de código

```
from kaggle_secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copia a la carpeta

GOOGLE_API_KEY

+ Añadir secreto Fragmento de copia

Explora las mejores prácticas para herramientas, incluyendo el uso de MCP y operaciones de larga duración.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Setup and authentication complete.

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit. This keeps your code organized and ensures we have access to the necessary building blocks.

```
[2]: import uuid
from google.genai import types

from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

from google.adk.tools.mcp_tool.mcp_toolset import McpToolset
from google.adk.tools.tool_context import ToolContext
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

from google.adk.apps.app import App, ResumabilityConfig
from google.adk.tools.function_tool import FunctionTool

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

+ Code + Markdown

1.4: Configure Botpy Options

Day 2b - Agent Tools Best Practices

The `McpToolset` is used to integrate an ADK Agent with an MCP Server.

What the code does:

- Uses `npx` (Node package runner) to run the MCP server
- Connects to `@modelcontextprotocol/server-everything`
- Filters to only use the `getTinyImage` tool (the server has others, but we only need this one)

```
# MCP integration with Everything Server
mcp_image_server = McpToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command="npx", # Run MCP server via npx
            args=[
                "-y", # Argument for npx to auto-confirm install
                "@modelcontextprotocol/server-everything",
            ],
            tool_filter=["getTinyImage"],
        ),
        timeout=30,
    )
)

print("✅ MCP Tool created")
```

✅ MCP Tool created

+ Code + Markdown

Behind the scenes:

Day 2b - Agent Tools Best Practices

```
# Create shipping agent with pausable tool
shipping_agent = LlmAgent(
    name="shipping_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="You are a shipping coordinator assistant."

    When users request to ship containers:
    1. Use the place_shipping_order tool with the number of containers and destination
    2. If the order status is 'pending', inform the user that approval is required
    3. After receiving the final result, provide a clear summary including:
       - Order status (approved/rejected)
       - Order ID (if available)
       - Number of containers and destination
    4. Keep responses concise but informative
    ...
    tools=[FunctionTool(func=place_shipping_order)],
)

print("✅ Shipping Agent created!")
```

✅ Shipping Agent created!

+ Code + Markdown

Step 2: Wrap in resumable App

The problem: A regular `LlmAgent` is stateless - each call is independent with no memory of previous interactions. If a tool requests approval, the agent can't remember what it was doing.

The solution: Wrap your agent in an `App` with **resumability enabled**. The App adds a persistence layer that saves and restores state.

What gets saved when a tool pauses:

- All conversation messages so far
- Which tool was called (`place_shipping_order`)

```
# Wrap the agent in a resumable app - THIS IS THE KEY FOR LONG-RUNNING OPERATIONS!
Execute cell. Cell execution started at 1:50am
name=shipping_coordinator",
root_agent=shipping_agent,
resumability_config=ResumabilityConfig(is_resumable=True),
)

print("✅ Resumable app created!")

✅ Resumable app created!
/tmp/ipykernel_49/367377575.py:5: UserWarning: [EXPERIMENTAL] ResumabilityConfig: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  resumability_config=ResumabilityConfig(is_resumable=True),
+ Code + Markdown
```

Step 3: Create Session and Runner with the App

```
session_service = InMemorySessionService()

# Create runner with the resumable app
shipping_runner = Runner(
    app=shipping_app, # Pass the app instead of the agent
    session_service=session_service,
)

print("✅ Runner created!")

✅ Runner created!
+ Code + Markdown

def create_approval_response(approval_info, approved):
    """Create approval response message."""
    confirmation_response = types.FunctionResponse(
        id=approval_info["approval_id"],
        name="adk_request_confirmation",
        response={"confirmed": approved},
    )
    return types.Content(
        role="user", parts=[types.Part(function_response=confirmation_response)]
    )

print("✅ Helper functions defined")

✅ Helper functions defined
+ Code + Markdown
```

4.4: The Workflow Function - Let's tie it all together!

```
Day 2b - Agent Tools Best Practices
Archivo Edición Vista Corre Escenarios Complementos Ayuda
Compartir Guardar la versión 0
Sesión de borrador (17m)
Ejecutar todo Código
print(f"👤 Human Decision: {'APPROVE ✅' if auto_approve else 'REJECT ❌'}\n")

# PATH A: Resume the agent by calling run_async() again with the approval decision
async for event in shipping_runner.run_async(
    user_id="test-user",
    session_id=session_id,
    new_message=create_approval_response(
        approval_info, auto_approve
    ), # Send human decision here
    invocation_id=approval_info["invocation_id"], # Critical: same invocation_id tells ADK to RESUME
):
    if event.content and event.content.parts:
        for part in event.content.parts:
            if part.text:
                print(f"Agent > {part.text}")

# -----
#
else:
    # PATH B: If the 'adk_request_confirmation' is not present - no approval needed - order completed immediately.
    print_agent_response(events)

print(f"{'='*60}\n")

print("✅ Workflow function ready")

✅ Workflow function ready
+ Code + Markdown
```

Grupos

Curso intensivo de a...

Day 2b - Agent Tools

Kaggle Notebook Edi...

Day 2a - Agent Tools

Kaggle Notebook Edi...

Herramientas de Age...

(157) Whitepaper Co...

https://www.kaggle.com/learn-guide/5-day-agents

Chat

+

Crear

Inicio

Competiciones

Referencias

Arena de Juego

Centro de datos

Más

Tu obra

Visto

Día 0 - Resolución de...

Día 1b - Arquitectura...

Día 1a - De la Consig...

Editado

Día 1b - Arquitectura...

Día 1a - De la Consig...

Día 1a - De la Consig...

Ver eventos activos

Búsqueda

Explorar y encontrar cosas para aprender, resolver y compartir con otros.

Día 2 (Herramientas de Agente e Interoperabilidad con el Protocolo de Contexto de Modelo (MCP))

Bienvenidos al Día 2.

Este documento técnico se centra en herramientas externas, funciones que permiten a un agente realizar acciones o recuperar datos en tiempo real más allá de su conjunto de entrenamiento e introduce las mejores prácticas para diseñar herramientas eficaces. Aprenderás sobre MCP, destacando sus componentes arquitectónicos, la capa de comunicación, los riesgos y las brechas de preparación empresarial.

En los codelabs, crearás herramientas personalizadas para tus agentes convirtiendo tus propias funciones en Python en acciones que tu agente pueda realizar. También usarás MCP e implementarás operaciones de larga duración donde un agente puede pausar las llamadas a herramientas mientras espera la aprobación humana, antes de reanudar.

Asignaciones del día 2:

Completar la Unidad 2 - "Herramientas de Agente e Interoperabilidad con el Protocolo de Contexto del Modelo (MCP)":

☒

Escucha el episodio resumen del podcast de esta unidad.

☒

Para complementar el pódcast, lee el [whitepaper "Agent Tools & Interoperability with Model Context Protocol \(MCP\)"](#).

☒

Completa estos codelabs en Kaggle:

☒

Explora nuevas formas de añadir herramientas que amplíen lo que tus agentes pueden hacer.

☒

Explora las mejores prácticas para herramientas, incluyendo el uso de MCP y operaciones de larga duración.

[Opcional]: Mira la retransmisión en directo grabada de YouTube. Kanchana Patilola y Anant Nawalgaria estuvieron acompañados por la autora de Codelab, Laxmi Harikumar, junto con los invitados de Google Edward Grefenstette, Mike Styer y Oriol Vinyals, así como el ponente externo Alex Wissner-Gross de [Reified](#), para debatir las asignaciones y compartir sus ideas.

Día 3 (Ingeniería de Contexto: Sesiones y Memoria)

Escucha el episodio resumen del podcast de esta unidad

The image shows a YouTube video player interface. The video title is "Whitepaper Companion Podcast". The video description includes the text "Podcast complementario del informe técnico - Ingeniería de contexto: Sesiones y memoria". The video has 66k views and was uploaded 1 month ago. The video player shows the video content, which is a podcast episode summary. The video player also shows the channel name "Kaggle" and the number of subscribers "324 k suscriptores". The video player has a search bar and a "Buscar" button. The video player also shows the video title "Whitepaper Companion Podcast" and the video description "Podcast complementario del informe técnico - Ingeniería de contexto: Sesiones y memoria". The video player also shows the video has 66k views and was uploaded 1 month ago. The video player also shows the channel name "Kaggle" and the number of subscribers "324 k suscriptores". The video player has a search bar and a "Buscar" button.

Para complementar el pódcast, lee el documento técnico "Ingeniería del contexto: Sesiones y memoria".

The image shows a Kaggle document viewer interface. The document title is "Ingeniería del Contexto: Sesiones y Memoria". The document authors are "Kimberly Milam y Antonio Gulli". The document introduction states: "Este documento blanco explora el papel fundamental de las Sesiones y la Memoria en la construcción de estados agentes inteligentes de LLM para empoderar a los desarrolladores a crear más potentes, personalizados y experiencias persistentes de IA. Permitir que los Grandes Modelos de Lenguaje (LLMs) recuerden, aprendan y Personaliza las interacciones, los desarrolladores deben ensamblar y gestionar la información dinámicamente dentro de su ventana de contexto—un proceso conocido como Ingeniería del Contexto." The document also includes a list of references: 14. <https://arxiv.org/pdf/2412.15266>, 15. <https://arxiv.org/pdf/2412.15266>, 16. <https://cloud.google.com/vertex-ai/generative-ai/docs/model-reference/inference#sample-requests-text-gen-multimodal-prompt>, 17. <https://cloud.google.com/vertex-ai/generative-ai/docs/agent-engine/memory-bank/generate-memories>, 18. <https://cloud.google.com/vertex-ai/generative-ai/docs/multimodal/control-generated-output>, 19. <https://cloud.google.com/agent-builder/agent-engine/memory-bank/set-up#memory-bank-config>, 20. <https://arxiv.org/html/2504.19413v1>, 21. <https://google.github.io/aidk-docs/tools/#how-agents-use-tools>. The document also shows a sidebar with navigation links: Inicio, Competiciones, Referencias, Arena de Juego, Centro de datos, Más, Tu obra, Visto, Día 2b - Mejores pr..., Día 2a - Herramienta..., Día 0 - Resolución de..., Día 1b - Arquitectura..., Día 1a - De la Consig..., Editado, Día 2b - Mejores pr..., Ver eventos activos.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Gemini API key setup complete.

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
[3]: from typing import Any, Dict

from google.adk.agents import Agent, LlmAgent
from google.adk.apps.app import App, EventsCompactionConfig
from google.adk.models.google_llm import Gemini
from google.adk.sessions import DatabaseSessionService
from google.adk.sessions import InMemorySessionService
from google.adk.runners import Runner
from google.adk.tools.tool_context import ToolContext
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

Grupos1Curso intensivo de agentes de IA xDay 3a - Agent Sessions xEditor de cuadernos Kaggle xAgent Development Kit (ADK) - A. xIngeniería del Contexto: Sesiones x+ - Chat

https://www.kaggle.com/code/fernandaximena/day-3a-agent-sessions/edit?fromFork=1

Day 3a - Agent Sessions

Archivo Edición Vista Corre Escenarios Complementos Ayuda

Compartir Guardar la versión 0

+ Ejecutar todo Código

Sesión de borrador Carrera8m

```
# Process queries if provided
if user_queries:
    # Convert single query to list for uniform processing
    if type(user_queries) == str:
        user_queries = [user_queries]

    # Process each query in the list sequentially
    for query in user_queries:
        print(f"\nUser > {query}")

        # Convert the query string to the ADK Content format
        query = types.Content(role="user", parts=[types.Part(text=query)])

        # Stream the agent's response asynchronously
        async for event in runner_instance.run_async(
            user_id=USER_ID, session_id=session.id, new_message=query
        ):
            # Check if the event contains valid content
            if event.content and event.content.parts:
                # Filter out empty or "None" responses before printing
                if (
                    event.content.parts[0].text != "None"
                    and event.content.parts[0].text
                ):
                    print(f"{MODEL_NAME} > ", event.content.parts[0].text)

            else:
                print("No queries!")

print("✅ Helper functions defined.")
```

✅ Helper functions defined.

Day 3a - Agent Sessions

Archivo Edición Vista Corre Escenarios Complementos Ayuda

Compartir

Guardar la versión 0

+ Ejecutar todo Código

Sesión de borrador Carrera8m

```
APP_NAME = "default" # Application
USER_ID = "default" # User
SESSION = "default" # Session

MODEL_NAME = "gemini-2.5-flash-lite"

# Step 1: Create the LLM Agent
root_agent = Agent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot", # Description of the agent's purpose
)

# Step 2: Set up Session Management
# InMemorySessionService stores conversations in RAM (temporary)
session_service = InMemorySessionService()

# Step 3: Create the Runner
runner = Runner(agent=root_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Stateful agent initialized!")
print(f"  - Application: {APP_NAME}")
print(f"  - User: {USER_ID}")
print(f"  - Using: {session_service.__class__.__name__}")
```

```
✅ Stateful agent initialized!
  - Application: default
  - User: default
  - Using: InMemorySessionService
```

+ Code + Markdown

2.5 Testing Our Stateful Agent

Now let's see the magic of sessions in action!

```
# Run a conversation with two queries in the same session
# Notice: Both queries are part of the SAME session, so context is maintained
await run_session(
    runner,
    [
        "Hi, I am Sam! What is the capital of United States?",
        "Hello! What is my name?", # This time, the agent should remember!
    ],
    "stateful-agentic-session",
)

### Session: stateful-agentic-session

User > Hi, I am Sam! What is the capital of United States?
gemini-2.5-flash-lite > Hi Sam! The capital of the United States is Washington, D.C.

User > Hello! What is my name?
gemini-2.5-flash-lite > Your name is Sam. You told me that at the beginning of our conversation!
```

+ Code + Markdown

● (Optional) 2.6 Testing Agent's forgetfulness

To verify that the agent forgets the conversation, **restart the kernel**. Then, **run ALL the previous cells in this notebook EXCEPT the `run_session` in 2.5**.

Now run the cell below. You'll see that the agent doesn't remember anything from the previous conversation.

```
# Run this cell after restarting the kernel. All this history will be gone...
await run_session(
    runner,
    ["What did I ask you about earlier?", "And remind me, what's my name?"],
    "stateful-agentic-session",
) # Note, we are using same session name
```

Session: stateful-agentic-session

User > What did I ask you about earlier?

gemini-2.5-flash-lite > You asked me about the capital of the United States earlier.

User > And remind me, what's my name?

gemini-2.5-flash-lite > Your name is Sam.

+ Code + Markdown

```
# Step 1: Create the same agent (notice we use LlmAgent this time)
chatbot_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot with persistent memory",
)

# Step 2: Switch to DatabaseSessionService
# SQLite database will be created automatically
db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Step 3: Create a new runner with persistent storage
runner = Runner(agent=chatbot_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Upgraded to persistent sessions!")
print(f" - Database: my_agent_data.db")
print(f" - Sessions will survive restarts!")
```

✅ Upgraded to persistent sessions!
- Database: my_agent_data.db
- Sessions will survive restarts!

+ Code + Markdown

3.3 Test Run 1: Verifying Persistence

In this first test run, we'll start a new conversation with the session ID `test-db-session-01`. We will first introduce our name as 'Sam' and then ask a question. In the second turn, we will ask the agent for our name.

Since we are using `DatabaseSessionService`, the agent should remember the name.

After the conversation, we'll inspect the `my_agent_data.db` SQLite database directly to see how the conversation `events` (the user queries and model responses) are stored.

+ Code + Markdown

```
await run_session(
    runner,
    ["Hi, I am Sam! What is the capital of the United States?", "Hello! What is my name?"],
    "test-db-session-01",
)
```

Session: test-db-session-01

User > Hi, I am Sam! What is the capital of the United States?

gemini-2.5-flash-lite > Hi Sam! The capital of the United States is Washington, D.C.

User > Hello! What is my name?

gemini-2.5-flash-lite > Your name is Sam.

● (Optional) 3.4 Test Run 2: Resuming a Conversation

Now, let's repeat the test again, but this time, **let's stop this Kaggle Notebook's kernel and restart it again.**

1. Run all the previous cells in the notebook, **EXCEPT** the previous Section 3.3 (`run_session` cell).
2. Now, run the below cell with the **same session ID** (`test-db-session-01`).

We will ask a new question and then ask for our name again. **Because the session is loaded from the database, the agent should still remember** that our name is 'Sam' from the first test run. This demonstrates the power of persistent sessions.

```
await run_session(
    runner,
    [{"What is the capital of India?", "Hello! What is my name?"},
     "test-db-session-01",
])
```

Session: test-db-session-01

User > What is the capital of India?
gemini-2.5-flash-lite > The capital of India is New Delhi.

User > Hello! What is my name?
gemini-2.5-flash-lite > Your name is Sam.

+ Code + Markdown

3.5 Let's verify that the session data is isolated

As mentioned earlier, a session is private conversation between an Agent and a User (i.e., two sessions do not share information). Let's run our `run_session` with a different session name `test-db-session-02` to confirm this.

```
await run_session(
    runner, [{"Hello! What is my name?"}, "test-db-session-02"]
) # Note, we are using new session name
```

Session: test-db-session-02

User > Hello! What is my name?
gemini-2.5-flash-lite > I do not have access to personal information about you, including your name. I am a text-based chatbot and do not collect or store any user data.

```
import sqlite3

def check_data_in_db():
    with sqlite3.connect("my_agent_data.db") as connection:
        cursor = connection.cursor()
        result = cursor.execute(
            "select app_name, session_id, author, content from events"
        )
        print([_[0] for _ in result.description])
        for each in result.fetchall():
            print(each)

check_data_in_db()
```

```
[ 'app_name', 'session_id', 'author', 'content' ]
('default', 'test-db-session-01', 'user', '{"parts": [{"text": "Hi, I am Sam! What is the capital of the United States?"}], "role": "user"}')
('default', 'test-db-session-01', 'text_chat_bot', '{"parts": [{"text": "Hi Sam! The capital of the United States is Washington, D.C."}], "role": "model"}')
('default', 'test-db-session-01', 'user', '{"parts": [{"text": "Hello! What is my name?"}], "role": "user"}')
('default', 'test-db-session-01', 'text_chat_bot', '{"parts": [{"text": "Your name is Sam."}], "role": "model"}')
('default', 'test-db-session-01', 'user', '{"parts": [{"text": "What is the capital of India?"}], "role": "user"}')
('default', 'test-db-session-01', 'text_chat_bot', '{"parts": [{"text": "The capital of India is New Delhi."}], "role": "model"}')
('default', 'test-db-session-01', 'user', '{"parts": [{"text": "Hello! What is my name?"}], "role": "user"}')
('default', 'test-db-session-01', 'text_chat_bot', '{"parts": [{"text": "Your name is Sam."}], "role": "model"}')
('default', 'test-db-session-02', 'user', '{"parts": [{"text": "Hello! What is my name?"}], "role": "user"}')
('default', 'test-db-session-02', 'text_chat_bot', '{"parts": [{"text": "I do not have access to personal information about you, including your name. I am a text-based chatbot and do not collect or store any user data."}], "role": "model"}')
```

+ Code + Markdown

```
# Re-define our app with Events Compaction enabled
research_app_compacting = App(
    name="research_app_compacting",
    root_agent=chatbot_agent,
    # This is the new part!
    events_compaction_config=EventsCompactionConfig(
        compaction_interval=3, # Trigger compaction every 3 invocations
        overlap_size=1, # Keep 1 previous turn for context
    ),
)

db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Create a new runner for our upgraded app
research_runner_compacting = Runner(
    app=research_app_compacting, session_service=session_service
)

print("✅ Research App upgraded with Events Compaction!")

✅ Research App upgraded with Events Compaction!
/tmp/ipykernel_48/3773147741.py:6: UserWarning: [EXPERIMENTAL] EventsCompactionConfig: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  events_compaction_config=EventsCompactionConfig(
+ Code + Markdown
```

Grupos 1

Curso intensivo de agentes de IA

Day 3a - Agent Sessions

Editor de cuadernos Kaggle

Agent Development Kit (ADK)

Ingeniería del Contexto: Sesiones

https://www.kaggle.com/code/fernandaximena/day-3a-agent-sessions/edit?fromfork=1

Compartir Guardar la versión 0

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ Ejecutar todo Código

Sesión de borrador Carrera15m

Session: compaction_demo

User > What is the latest news about AI in healthcare?

gemini-2.5-flash-lite > The field of AI in healthcare is constantly evolving, so "latest" can change rapidly! However, here are some of the key trends and exciting developments that have been making headlines recently and continue to be areas of active research and implementation:

1. Advancements in Diagnostics and Imaging:

* **Improved Accuracy and Speed:** AI algorithms are getting remarkably good at analyzing medical images (X-rays, CT scans, MRIs, pathology slides) to detect diseases like cancer, diabetic retinopathy, and cardiovascular conditions with high accuracy, sometimes even surpassing human radiologists in specific tasks. This can lead to earlier diagnosis and better patient outcomes.

* **Assisted Diagnosis:** AI isn't necessarily replacing doctors but acting as a powerful assistant, flagging potential areas of concern for clinicians to review, reducing missed diagnoses, and speeding up the diagnostic process.

* **Personalized Screening:** AI can help tailor screening recommendations based on individual risk factors, moving away from a one-size-fits-all approach.

2. Drug Discovery and Development:

Day 3a - Agent Sessions

```
# Get the final session state
final_session = await session_service.get_session(
    app_name=research_runner_compacting.app_name,
    user_id=USER_ID,
    session_id="compaction_demo",
)

print("--- Searching for Compaction Summary Event ---")
found_summary = False
for event in final_session.events:
    # Compaction events have a 'compaction' attribute
    if event.actions and event.actions.compaction:
        print("\n✅ SUCCESS! Found the Compaction Event:")
        print(f"  Author: {event.author}")
        print(f"  Compacted information: {event}")
        found_summary = True
        break

if not found_summary:
    print(
        "\n❌ No compaction event found. Try increasing the number of turns in the demo."
    )

--- Searching for Compaction Summary Event ---

✅ SUCCESS! Found the Compaction Event:
Author: user

Compacted information: model_version=None content=None grounding_metadata=None partial=None turn_complete=None finish_reason=None error_code=None error_message=None in
terrupted=None custom_metadata=None usage_metadata=None live_session_resumption_update=None input_transcription=None output_transcription=None avg_logprobs=None logprob
s_result=None cache_metadata=None citation_metadata=None invocation_id="e1567611-0f1a-4e63-ad7a-bab01e662490" author="user" actions=EventActions(skip_summarization=Non
e, state_delta={}, artifact_delta={}, transfer_to_agent=None, escalate=None, requested_auth_configs={}, requested_tool_confirmations={}, compaction={"start_timestamp":
1777959694.827739, "end_timestamp": 1777959788.415443, "compacted_content": {'parts': [{'function_call': None, 'code_execution_result': None, 'executable_code': None,
```

Day 3a - Agent Sessions

```
def save_userinfo(
    tool_context: ToolContext, user_name: str, country: str
) -> Dict[str, Any]:
    """
    Tool to record and save user name and country in session state.

    Args:
        user_name: The username to store in session state
        country: The name of the user's country
    """
    # Write to session state using the 'user:' prefix for user data
    tool_context.state["user:name"] = user_name
    tool_context.state["user:country"] = country

    return {"status": "success"}

# This demonstrates how tools can read from session state.
def retrieve_userinfo(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Tool to retrieve user name and country from session state.

    # Read from session state
    user_name = tool_context.state.get("user:name", "Username not found")
    country = tool_context.state.get("user:country", "Country not found")

    return {"status": "success", "user_name": user_name, "country": country}

print("✅ Tools created.")

✅ Tools created.
```

5.2 Creating an Agent with Session State Tools

Now let's create a new agent that has access to our session state management tools:

```
# Configuration
APP_NAME = "default"
USER_ID = "default"
MODEL_NAME = "gemini-2.5-flash-lite"

# Create an agent with session state tools
root_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot.
    Tools for managing user context:
    * To record username and country when provided use 'save_userinfo' tool.
    * To fetch username and country when required use 'retrieve_userinfo' tool.
    """,
    tools=[save_userinfo, retrieve_userinfo], # Provide the tools to the agent
)

# Set up session service and runner
session_service = InMemorySessionService()
runner = Runner(agent=root_agent, session_service=session_service, app_name="default")

print("✅ Agent with session state tools initialized!")
```

✅ Agent with session state tools initialized!

5.3 Testing Session State in Action

Let's test how the agent uses session state to remember information across conversation turns:

```
# Test conversation demonstrating session state
await run_session(
    runner,
    [
        "Hi there, how are you doing today? What is my name?", # Agent shouldn't know the name yet
        "My name is Sam. I'm from Poland.", # Provide name - agent should save it
        "What is my name? Which country am I from?", # Agent should recall from session state
    ],
    "state-demo-session",
)
```

Session: state-demo-session

User > Hi there, how are you doing today? What is my name?
gemini-2.5-flash-lite > Hello! I'm doing great, thank you for asking. I'm not sure what your name is, though. Would you like to tell me?

User > My name is Sam. I'm from Poland.

WARNING:google_genai.types:warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full content with `response.content.parts` accessor to get the full model response.

gemini-2.5-flash-lite > It is a pleasure to meet you, Sam! I will save your information.

User > What is my name? Which country am I from?
gemini-2.5-flash-lite > I have you recorded as Sam from Poland.

5.4 Inspecting Session State

Let's directly inspect the session state to see what's stored:

```
# Retrieve the session and inspect its state
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="state-demo-session"
)

print("Session State Contents:")
print(session.state)
print("\n🔍 Notice the 'user:name' and 'user:country' keys storing our data!")
```

Session State Contents:
{'user:name': 'Sam', 'user:country': 'Poland'}

🔍 Notice the 'user:name' and 'user:country' keys storing our data!

+ Code

+ Markdown

5.5 Session State Isolation

As we've already seen, an important characteristic of session state is that it's isolated per session. Let's demonstrate this by starting a new session:

```
# Start a completely new session - the agent won't know our name
await run_session(
    runner,
    ["Hi there, how are you doing today? What is my name?"],
    "new-isolated-session",
)

# Expected: The agent won't know the name because this is a different session
```

Session: new-isolated-session

User > Hi there, how are you doing today? What is my name?
gemini-2.5-flash-lite > Hello! I'm doing great, thank you for asking. I can't seem to recall your name at the moment. Could you please tell me what it is?

5.6 Cross-Session State Sharing

While sessions are isolated by default, you might notice something interesting. Let's check the state of our new session (`new-isolated-session`):

```
# Check the state of the new session
ice.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="new-isolated-session"
)

print("New Session State:")
print(session.state)

# Note: Depending on implementation, you might see shared state here.
# This is where the distinction between session-specific and user-specific state becomes important.
```

New Session State:
{'user:name': 'Sam', 'user:country': 'Poland'}

+ Code + Markdown

Cleanup

```
# Clean up any existing database to start fresh (if Notebook is restarted)
import os

if os.path.exists("my_agent_data.db"):
    os.remove("my_agent_data.db")
print("✅ Cleaned up old database files")
```

✅ Cleaned up old database files

+ Code + Markdown

Explora cómo usar la memoria con tu agente.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🚨 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Gemini API key setup complete.

+ Code + Markdown

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

```
from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from google.adk.tools import load_memory, preload_memory
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

+ Code + Markdown

The screenshot shows a web browser window with multiple tabs. The active tab is 'Kaggle Notebook Editor'. The address bar shows the URL: <https://www.kaggle.com/code/fernandaximena/day-3b-agent-memory/edit>. The notebook title is 'Day 3b - Agent Memory'. The code editor shows the following code:

```
session = await session_service.create_session(
    app_name=APP_NAME, user_id=USER_ID, session_id=session_id
)
except:
    session = await session_service.get_session(
        app_name=APP_NAME, user_id=USER_ID, session_id=session_id
    )

# Convert single query to list
if isinstance(user_queries, str):
    user_queries = [user_queries]

# Process each query
for query in user_queries:
    print(f"\nUser > {query}")
    query_content = types.Content(role="user", parts=[types.Part(text=query)])

# Stream agent response
async for event in runner_instance.run_async(
    user_id=USER_ID, session_id=session_id, new_message=query_content
):
    if event.is_final_response() and event.content and event.content.parts:
        text = event.content.parts[0].text
        if text and text != "None":
            print(f"Model: > {text}")

print("✅ Helper functions defined.")
```

✅ Helper functions defined.

+ Code + Markdown

1.5: Configure Retrv Options

3.2 Add Memory to Agent

Next, create a simple agent to answer user queries.

```
# Define constants used throughout the notebook
APP_NAME = "MemoryDemoApp"
USER_ID = "demo_user"

# Create agent
user_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="MemoryDemoAgent",
    instruction="Answer user questions in simple words.",
)

print("✅ Agent created")
```

✅ Agent created

+ Code + Markdown

```
# Create Session Service
session_service = InMemorySessionService() # Handles conversations

# Create runner with BOTH services
runner = Runner(
    agent=user_agent,
    app_name="MemoryDemoApp",
    session_service=session_service,
    memory_service=memory_service, # Memory service is now available!
)

print("✅ Agent and Runner created with memory support!")
```

✅ Agent and Runner created with memory support!

+ Code + Markdown

```
# Search for color preferences
search_response = await memory_service.search_memory(
    app_name=APP_NAME, user_id=USER_ID, query="What is the user's favorite color?"
)

print("🔍 Search Results:")
print(f"Found {len(search_response.memories)} relevant memories")
print()

for memory in search_response.memories:
    if memory.content and memory.content.parts:
        text = memory.content.parts[0].text[:80]
        print(f" [{memory.author}]: {text}...")
```

🔍 Search Results:
Found 0 relevant memories

```
async def auto_save_to_memory(callback_context):
    """Automatically save session to memory after each agent turn."""
    await callback_context._invocation_context.memory_service.add_session_to_memory(
        callback_context._invocation_context.session
    )

print("✅ Callback created.")
```

✅ Callback created.

+ Code + Markdown

```
# Agent with automatic memory saving
auto_memory_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="AutoMemoryAgent",
    instruction="Answer user questions.",
    tools=[preload_memory],
    after_agent_callback=auto_save_to_memory, # Saves after each turn!
)

print("✅ Agent created with automatic memory saving!")
```

✅ Agent created with automatic memory saving!

+ Code + Markdown

```
# Create a runner for the auto-save agent
# This connects our automated agent to the session and memory services
auto_runner = Runner(
    agent=auto_memory_agent, # Use the agent with callback + preload_memory
    app_name=APP_NAME,
    session_service=session_service, # Same services from Section 3
    memory_service=memory_service,
)

print("✅ Runner created.")
```

✅ Runner created.

+ Code + Markdown

```
# Test 1: Tell the agent about a gift (first conversation)
# The callback will automatically save this to memory when the turn completes
await run_session(
    auto_runner,
    "I gifted a new toy to my nephew on his 1st birthday!",
    "auto-save-test",
)

# Test 2: Ask about the gift in a NEW session (second conversation)
# The agent should retrieve the memory using preload_memory and answer correctly
await run_session(
    auto_runner,
    "What did I gift my nephew?",
    "auto-save-test-2", # Different session ID - proves memory works across sessions!
)

### Session: auto-save-test

User > I gifted a new toy to my nephew on his 1st birthday!
```

+ Code + Markdown



Día 3 (Ingeniería de Contexto: Sesiones y Memoria)

Bienvenidos al Día 3.

Este whitepaper explora la ingeniería del contexto como la práctica de ensamblar y gestionar dinámicamente la información dentro de la ventana contextual de un agente para crear experiencias AI con estado y personalizadas. Define Sesiones como el contenedor para la historia de una conversación inmediata y única, y Memoria como el mecanismo de persistencia a largo plazo.

En los codelabs, aprenderás a hacer que los agentes tengan estado gestionando el historial de conversaciones mediante ingeniería de contexto en ADK y memoria de trabajo dentro de una sesión, permitiendo que tu agente recuerde el contexto y mantenga conversaciones coherentes de varios turnos. En el segundo cuaderno, le darás a tu agente memoria a largo plazo que persiste en diferentes sesiones.

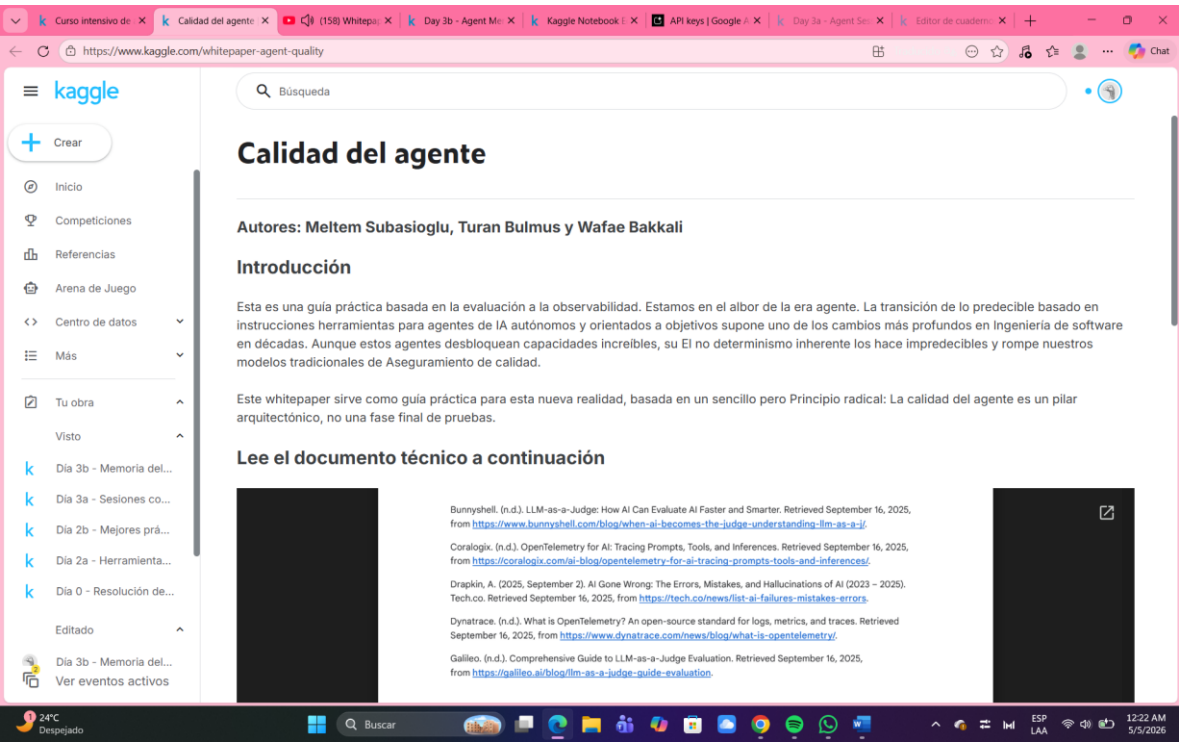
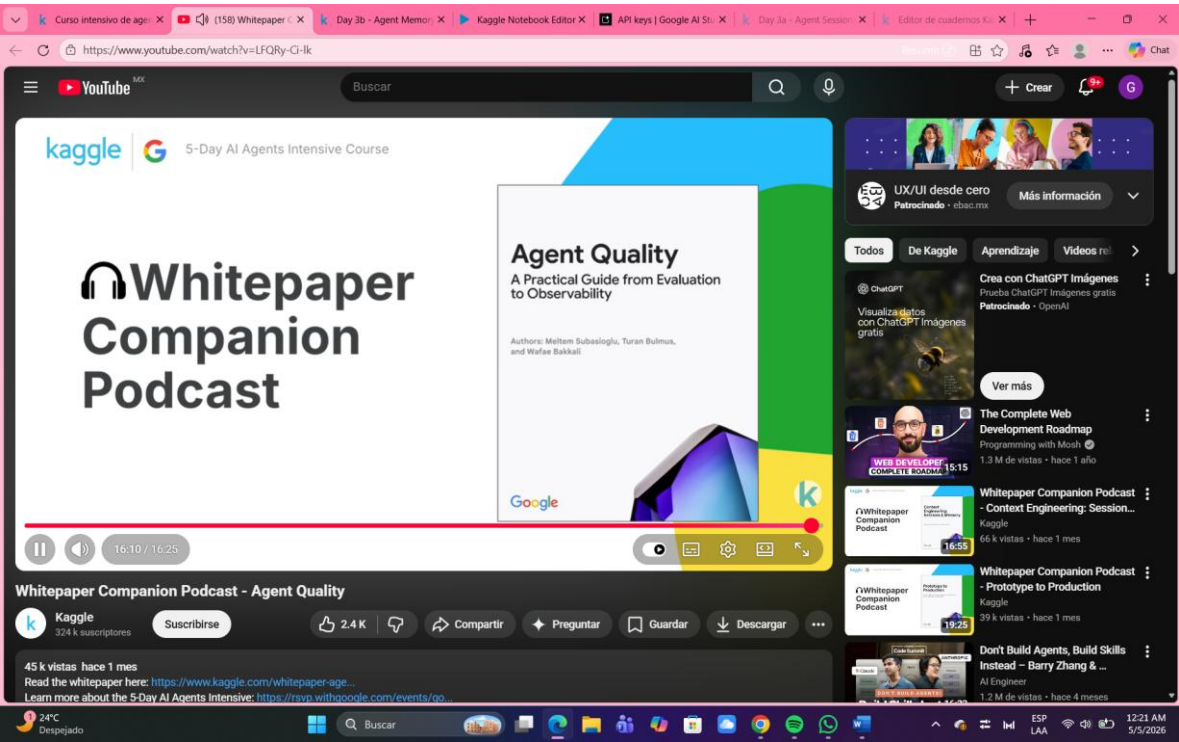
Día 3 Trabajo

Completar la Unidad 3 - "Ingeniería del Contexto: Sesiones y Memoria":

- ✅ Escucha el [episodio resumen del podcast](#) de esta unidad.
- ✅ Para complementar el podcast, lee el [documento técnico "Ingeniería del contexto: Sesiones y memoria"](#).
- ✅ Completa estos codelabs en Kaggle:
 - ✅ [Construye agentes con estado y realiza ingeniería de contexto.](#)
 - ✅ [Explora cómo usar la memoria con tu agente.](#)

[Opcional]: Mira la [retransmisión en directo grabada de YouTube](#). Kanchana Patlolla y Anant Nawalgaria estuvieron acompañadas por el autor de Codelab, Kristopher Overholt, junto con invitados de Google: Steven Johnson, Kimberly Milam y Julia Wiesinger, y el ponente externo Jay Alammar de [Cohere](#).

Día 4 (Calidad de agente)



```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(
        f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets"
    )
```

✅ Setup and authentication complete.

+ Code + Markdown

Curso intensivo de agentes de IA × Día 4a - Observabilidad del agente × Kaggle Notebook Editor × Day 3b - Agent Memory × Kaggle Notebook Editor

https://www.kaggle.com/code/fernandaximena/day-4a-agent-observability/edit

Day 4a - Agent Observability

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ ↻ 🔍 📄 📁 ▶ ⏮ Ejecutar todo Código Sesión de borrador Carrera2m

Col

```
<div style="padding: 15px; border: 2px solid #f0ad4e; border-radius: 8px; background-color: #f9f9f9;">
  <div style="font-family: sans-serif; margin-bottom: 12px; color: #333; font-size: 1.1em;">
    <strong>⚠ IMPORTANT: Action Required</strong>
  </div>
  <div style="font-family: sans-serif; margin-bottom: 15px; color: #333; line-height: 1.5;">
    The ADK web UI is <strong>not running yet</strong>. You must start it in the next cell.
    <ol style="margin-top: 10px; padding-left: 20px;">
      <li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code>
      <li style="margin-bottom: 5px;">Wait for that cell to show it is "Running" (it will not
      <li>Once it's running, <strong>return to this button</strong> and click it to open the
    </ol>
    <em style="font-size: 0.9em; color: #555;">(If you click the button before running the next
  </div>
  <a href='{url}' target='_blank' style="
    display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
    text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
    Open ADK Web UI (after running cell below) ↗
  </a>
</div>
<div>

display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")
```

✅ Helper functions defined.

+ Code + Markdown

Curso intensivo de agentes de IA | Día 4a - Observabilidad del agente | Kaggle Notebook Editor | Day 3b - Agent Memory | Kaggle Notebook Editor

https://www.kaggle.com/code/fernandaximena/day-4a-agent-observability/edit

Day 4a - Agent Observability

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ Ejecutar todo Código Sesión de borrador Carrera5m

2.1: Create a "Research Paper Finder" Agent

Our goal: Build a research paper finder agent that helps users find academic papers on any topic.

But first, let's intentionally create an incorrect version of the agent to practice debugging! We'll start by creating a new agent folder using the `!adk create` CLI command.

```
!adk create research-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

Agent created in `/kaggle/working/research-agent`:

- `.env`
- `__init__.py`
- `agent.py`

+ Code + Markdown

Agent definition

Curso intensivo de agentes de IA | Día 4a - Observabilidad del agente | Kaggle Notebook Editor | Day 3b - Agent Memory | Kaggle Notebook Editor

https://www.kaggle.com/code/fernandaximena/day-4a-agent-observability/edit

Day 4a - Agent Observability

Draft guardado

Archivo Edición Vista Corre Escenarios Complementos Ayuda

+ Ejecutar todo Código Sesión de borrador Carrera5m

```
Returns:
    The number of papers in the list.
    """
    return len(papers)

google_search_agent
google_search_agent = LlmAgent(
    name="google_search_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    description="Searches for information using Google search",
    instruction="""Use the google_search tool to find information on the given topic. Return the raw search results. If the user asks for a list of papers, then give them the list of research papers you found and not the raw search results.""",
    tools=[google_search]

)

root_agent
root_agent = LlmAgent(
    name="research_paper_finder_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="""Your task is to find research papers and count them.

    You MUST ALWAYS follow these steps:
    1) Find research papers on the user provided topic using the 'google_search_agent'.
    2) Then, pass the papers to 'count_papers' tool to count the number of papers returned.
    3) Return both the list of research papers and the total number of papers.
    """,
    tools=[AgentTool(agent=google_search_agent), count_papers]

)
```

Overwriting research-agent/agent.py

+ Code + Markdown

Com Sec

Frag

frd

USE

sec

+

Get the proxied URL to access the ADK web UI in the Kaggle Notebooks environment:

+ Code

+ Markdown



```
url_prefix = get_adk_proxy_url()
```

⚠️ IMPORTANT: Action Required

The ADK web UI is **not running yet**. You must start it in the next cell.

1. **Run the next cell** (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, **return to this button** and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

Open ADK Web UI (after running cell below) ↗

+ Code

+ Markdown

🔥 1.3: Set up logging and cleanup old files

Let's configure logging for our debugging session. The following cell makes sure we also capture other log levels, like DEBUG.



```
import logging
import os

# Clean up any previous logs
for log_file in ["logger.log", "web.log", "tunnel.log"]:
    if os.path.exists(log_file):
        os.remove(log_file)
        print(f"🧹 Cleaned up {log_file}")

# Configure logging with DEBUG log level.
logging.basicConfig(
    filename="logger.log",
    level=logging.DEBUG,
    format="%(filename)s:%(lineno)s %(levelname)s:%(message)s",
)

print("✅ Logging configured")
```

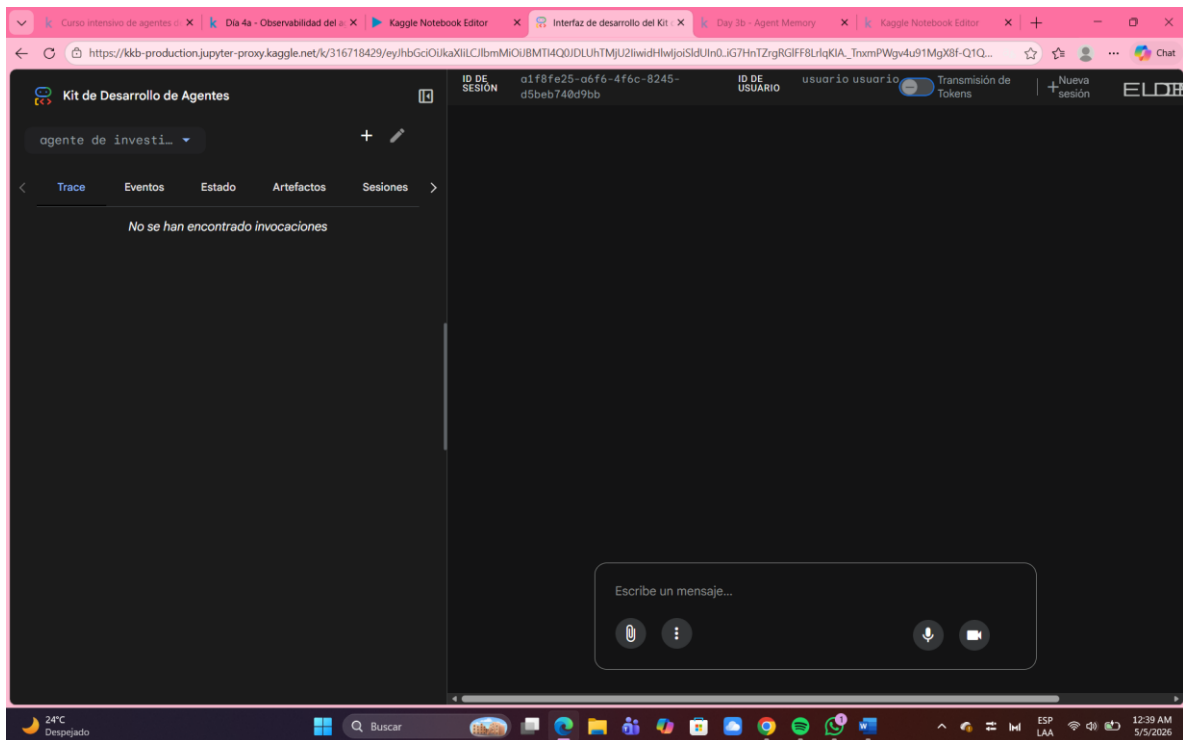
✅ Logging configured

+ Code

+ Markdown

```
!adk web --log_level DEBUG --url_prefix {url_prefix}

INFO: 35.191.18.194:0 - "GET /apps/research-agent/dev/ui/assets/runtime-config.json HTTP/1.1" 304 Not Modified
INFO: 35.191.18.199:0 - "GET /debug/trace/session/a1f8fe25-a6f6-4f6c-8245-d5beb740d9bb HTTP/1.1" 200 OK
INFO: 35.191.79.230:0 - "GET /apps/research-agent/eval_results HTTP/1.1" 200 OK
INFO: 35.191.18.200:0 - "GET /apps/research-agent/eval_sets HTTP/1.1" 200 OK
INFO: 35.191.18.192:0 - "GET /apps/research-agent/users/user/sessions HTTP/1.1" 200 OK
INFO: 35.191.17.52:0 - "GET /dev-ui/?app=research-agent&session=a1f8fe25-a6f6-4f6c-8245-d5beb740d9bb&userId=user HTTP/1.1" 200 OK
INFO: 35.191.121.122:0 - "GET /dev-ui/assets/config/runtime-config.json HTTP/1.1" 304 Not Modified
INFO: 35.191.121.120:0 - "GET /list-apps?relative_path=./ HTTP/1.1" 200 OK
INFO: 35.191.121.123:0 - "GET /builder/app/research-agent?ts=1777962920701 HTTP/1.1" 200 OK
INFO: 35.191.16.161:0 - "GET /builder/app/research-agent?ts=1777962920702 HTTP/1.1" 200 OK
INFO: 35.191.121.121:0 - "GET /apps/research-agent/users/user/sessions/a1f8fe25-a6f6-4f6c-8245-d5beb740d9bb HTTP/1.1" 200 OK
INFO: 35.191.121.122:0 - "GET /debug/trace/session/a1f8fe25-a6f6-4f6c-8245-d5beb740d9bb HTTP/1.1" 200 OK
INFO: 35.191.16.162:0 - "GET /apps/research-agent/eval_sets HTTP/1.1" 200 OK
INFO: 35.191.121.120:0 - "GET /apps/research-agent/eval_results HTTP/1.1" 200 OK
INFO: 35.191.16.160:0 - "GET /apps/research-agent/users/user/sessions HTTP/1.1" 200 OK
```



Curso intensivo de agentes x Día 4a - Observabilidad del x Kaggle Notebook Editor x Interfaz de desarrollo del Kit x Day 3b - Agent Memory x Kaggle Notebook Editor x + - Chat

https://www.kaggle.com/code/fernandaximena/day-4a-agent-observability/edit

Day 4a - Agent Observability

Archivo Edición Vista | Corre Escenarios Complementos Ayuda

Ejecutar todo Código Sesión de borrador Carrera14m

```
from google.adk.agents.callback_context import CallbackContext
from google.adk.models.llm_request import LlmRequest
from google.adk.plugins.base_plugin import BasePlugin

# Applies to all agent and model calls
class CountInvocationPlugin(BasePlugin):
    """A custom plugin that counts agent and tool invocations."""

    def __init__(self) -> None:
        """Initialize the plugin with counters."""
        super().__init__(name="count_invocation")
        self.agent_count: int = 0
        self.tool_count: int = 0
        self.llm_request_count: int = 0

    # Callback 1: Runs before an agent is called. You can add any custom logic here.
    async def before_agent_callback(
        self, *, agent: BaseAgent, callback_context: CallbackContext
    ) -> None:
        """Count agent runs."""
        self.agent_count += 1
        logging.info(f"[Plugin] Agent run count: {self.agent_count}")

    # Callback 2: Runs before a model is called. You can add any custom logic here.
    async def before_model_callback(
        self, *, callback_context: CallbackContext, llm_request: LlmRequest
    ) -> None:
        """Count LLM requests."""
        self.llm_request_count += 1
        logging.info(f"[Plugin] LLM request count: {self.llm_request_count}")

    ----- EXAMPLE PLUGIN - DOES NOTHING -----
```

Compartir Guardar la versión 0

Secretos

Fragmento de código

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copia a la carpeta

☒ GOOGLE_API_KEY

+ Añadir secreto Fragmento de copia

24°C Despejado 12:40 AM 5/5/2026

Curso intensivo de agentes x Día 4a - Observabilidad del x Kaggle Notebook Editor x Interfaz de desarrollo del Kit x Day 3b - Agent Memory x Kaggle Notebook Editor x + - Chat

https://www.kaggle.com/code/fernandaximena/day-4a-agent-observability/edit

Day 4a - Agent Observability

Archivo Edición Vista | Corre Escenarios Complementos Ayuda

Ejecutar todo Código Sesión de borrador Carrera14m

```
# Google search agent
google_search_agent = LlmAgent(
    name="google_search_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    description="Searches for information using Google search",
    instruction="Use the google_search tool to find information on the given topic. Return the raw search results",
    tools=[google_search],
)

# Root agent
research_agent_with_plugin = LlmAgent(
    name="research_paper_finder_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="Your task is to find research papers and count them.",
    tools=[AgentTool(agent=google_search_agent), count_papers],
)

print("✅ Agent created")
```

Agent created

+ Code + Markdown

3.4: Add LoggingPlugin to Runner

The following code creates the `InMemoryRunner`. This is used to programmatically invoke the agent.

Compartir Guardar la versión 0

Secretos

Fragmento de código

```
from kaggle.secrets import UserSecrets
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret_value_0
```

Copia a la carpeta

☒ GOOGLE_API_KEY

+ Añadir secreto Fragmento de copia

24°C Despejado 12:40 AM 5/5/2026

```
from google.adk.runners import InMemoryRunner
from google.adk.plugins.logging_plugin import (
    LoggingPlugin,
) # <---- 1. Import the Plugin
from google.genai import types
import asyncio

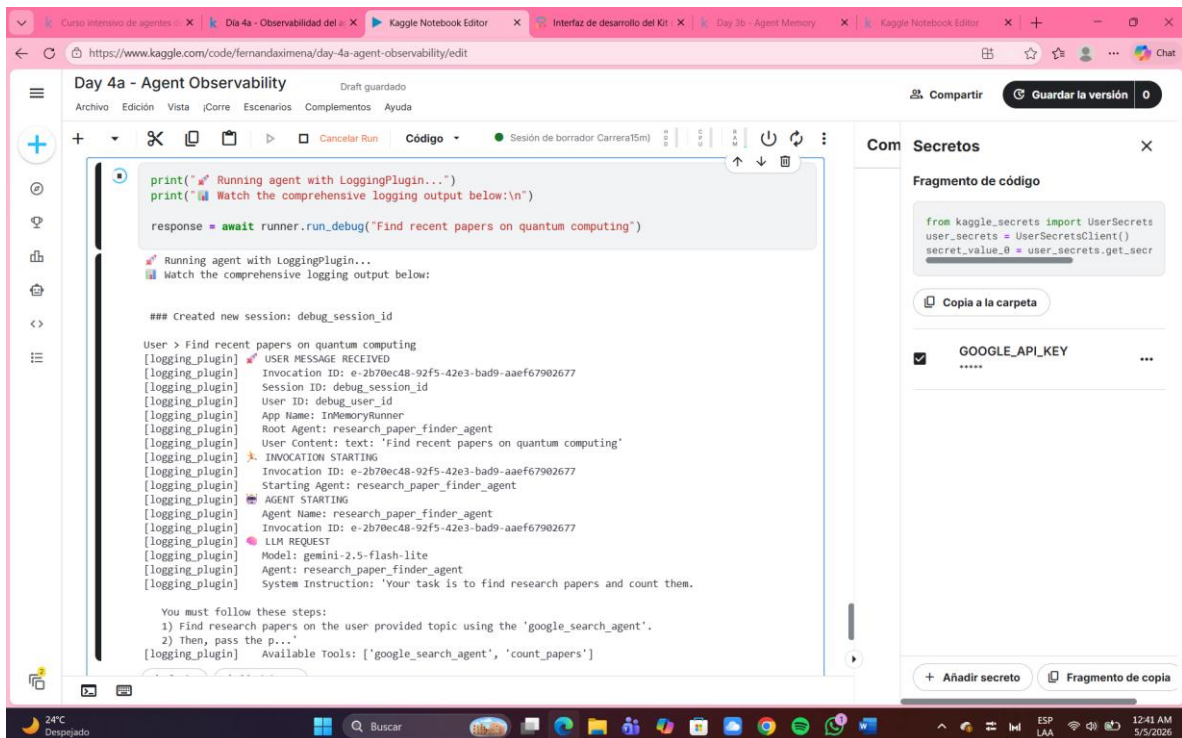
runner = InMemoryRunner(
    agent=research_agent_with_plugin,
    plugins=[
        LoggingPlugin()
    ], # <---- 2. Add the plugin. Handles standard Observability logging across ALL agents
)

print("✅ Runner configured")
```

✅ Runner configured

+ Code + Markdown

Let's now run the agent using `run_debug` function.



Evalúa a tus agentes.

3. Authenticate in the notebook

Run the cell below to access the `GOOGLE_API_KEY` you just saved and set it as an environment variable for the notebook to use:

```
[1]: import os
      from kaggle_secrets import UserSecretsClient

      try:
          GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
          os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
          print("✅ Setup and authentication complete.")
      except Exception as e:
          print(
              f"🔑 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets"
          )
```

✅ Setup and authentication complete.

```
        <li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code>
        <li style="margin-bottom: 5px;">Wait for that cell to show it is "Running" (it will not
        <li>Once it's running, <strong>return to this button</strong> and click it to open the
    </ol>
    <em style="font-size: 0.9em; color: #555;">(If you click the button before running the next
</div>
<a href='{url}' target='_blank' style="
    display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
    text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
    Open ADK Web UI (after running cell below) ↗
</a>
</div>
"""

display(HTML(styled_html))

return url_prefix
```

```
print("✅ Helper functions defined.")
```

✅ Helper functions defined.

+ Code

+ Markdown

Section 2: Create a Home Automation Agent

Let's create the agent that will be the center of our evaluation story. This home automation agent seems perfect in basic tests but has hidden flaws we'll discover through comprehensive evaluation. Run the `!adk create` CLI command to set up the project scaffolding.

+ Code

+ Markdown

▶

```
!adk create home_automation_agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

↑

↓

🗑

Agent created in /kaggle/working/home_automation_agent:
- .env
- __init__.py
- agent.py

+ Code

+ Markdown

Run the below cell to create the home automation agent.

+ Code

+ Markdown

▶

```
Returns:
    A dictionary confirming the action.
"""
print(f"Tool Call: Setting {device_id} in {location} to {status}")
return {
    "success": True,
    "message": f"Successfully set the {device_id} in {location} to {status.lower()}."
}

# This agent has DELIBERATE FLAWS that we'll discover through evaluation!
root_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="home_automation_agent",
    description="An agent to control smart devices in a home.",
    instruction="""You are a home automation assistant. You control ALL smart devices in the house.

    You have access to lights, security systems, ovens, fireplaces, and any other device the user menti
    Always try to be helpful and control whatever device the user asks for.

    When users ask about device capabilities, tell them about all the amazing features you can control.
    tools=[set_device_status],
)

Overwriting home_automation_agent/agent.py
```

↑

↓

🗑

+ Code

+ Markdown

+ Code

+ Markdown

▶

```
url_prefix = get_adk_proxy_url()
```

↑

↓

🗑

⚠ IMPORTANT: Action Required

The ADK web UI is **not running yet**. You must start it in the next cell.

1. **Run the next cell** (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, **return to this button** and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

Open ADK Web UI (after running cell below) ↗

+ Code

+ Markdown



```
!adb web --url_prefix {{url_prefix}}
```



```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
```

```
credential_service = InMemoryCredentialService()
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
```

```
super().__init__()
```

```
INFO: Started server process [90]
```

```
INFO: Waiting for application startup.
```

```
+-----+
| ADK Web Server started                |
|                                     |
| For local testing, access at http://127.0.0.1:8000. |
+-----+
```

```
INFO: Application startup complete.
```



```

import json

with open("home_automation_agent/integration.evalset.json", "w") as f:
    json.dump(test_cases, f, indent=2)

print("✅ Evaluation test cases created")
print("\n🔧 Test scenarios:")
for case in test_cases["eval_cases"]:
    user_msg = case["conversation"][0]["user_content"]["parts"][0]["text"]
    print(f"• {case['eval_id']}: {user_msg}")

print("\n📋 Expected results:")
print("• basic_device_control: Should pass both criteria")
print(
    "• wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters"
)
print(
    "• poor_response_quality_test: May fail response_match if response differs too much"
)

```

✅ Evaluation test cases created

🔧 Test scenarios:

- living_room_light_on: Please turn on the floor lamp in the living room
- kitchen_on_off_sequence: Switch on the main light in the kitchen.

📋 Expected results:

- basic_device_control: Should pass both criteria
- wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters
- poor_response_quality_test: May fail response_match if response differs too much

+ Code

+ Markdown

```

print("🔧 Run this command to execute evaluation:")
!adk eval home_automation_agent home_automation_agent/integration.evalset.json --config_file_path=home_

```

🔧 Run this command to execute evaluation:

```

/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/metric_evaluator_registry.py:90: UserWarning: [EXPERIMENTAL] MetricEvaluatorRegistry: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  metric_evaluator_registry = MetricEvaluatorRegistry()
/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/local_eval_service.py:79: UserWarning: [EXPERIMENTAL] UserSimulatorProvider: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  user_simulator_provider = UserSimulatorProvider()
Using evaluation criteria: criteria={'tool_trajectory_avg_score': 1.0, 'response_match_score': 0.8} user_simulator_config=None
/usr/local/lib/python3.11/dist-packages/google/adk/cli/cli_tools_click.py:650: UserWarning: [EXPERIMENTAL] UserSimulatorProvider: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  user_simulator_provider = UserSimulatorProvider()
/usr/local/lib/python3.11/dist-packages/google/adk/cli/cli_tools_click.py:655: UserWarning: [EXPERIMENTAL] LocalEvalService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  eval_service = LocalEvalService()
/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/user_simulator_provider.py:77: UserWarning: [EXPERIMENTAL] StaticUserSimulator: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  return StaticUserSimulator(static_conversation=eval_case.conversation)
/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/static_user_simulator.py:39: UserWarning: [EXPERIMENTAL] UserSimulator: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO:google_adk.google.adk.plugins.plugin_manager:Plugin 'request_interceptor_plugin' registered.
INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLLMVariant.GEMINI_API, stream: False
INFO:google_adk.google.adk.plugins.plugin_manager:Plugin 'request_interceptor_plugin' registered.
INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLL

```

12]:

```
# Analyzing evaluation results - the data science approach
print("📊 Understanding Evaluation Results:")
print()
print("🔍 EXAMPLE ANALYSIS:")
print()
print("Test Case: living_room_light_on")
print("❌ response_match_score: 0.45/0.80")
print("✅ tool_trajectory_avg_score: 1.0/1.0")
print()
print("📝 What this tells us:")
print("• TOOL USAGE: Perfect - Agent used correct tool with correct parameters")
print("• RESPONSE QUALITY: Poor - Response text too different from expected")
print("• ROOT CAUSE: Agent's communication style, not functionality")
print()
print("💡 ACTIONABLE INSIGHTS:")
print("1. Technical capability works (tool usage perfect)")
print("2. Communication needs improvement (response quality failed)")
print("3. Fix: Update agent instructions for clearer language or constrained response.")
print()
```

📊 Understanding Evaluation Results:

🔍 EXAMPLE ANALYSIS:

Test Case: living_room_light_on

❌ response_match_score: 0.45/0.80
✅ tool_trajectory_avg_score: 1.0/1.0

📝 What this tells us:

- TOOL USAGE: Perfect - Agent used correct tool with correct parameters
- RESPONSE QUALITY: Poor - Response text too different from expected
- ROOT CAUSE: Agent's communication style, not functionality

📊 Understanding Evaluation Results:

🔍 EXAMPLE ANALYSIS:

Test Case: living_room_light_on

❌ response_match_score: 0.45/0.80
✅ tool_trajectory_avg_score: 1.0/1.0

📝 What this tells us:

- TOOL USAGE: Perfect - Agent used correct tool with correct parameters
- RESPONSE QUALITY: Poor - Response text too different from expected
- ROOT CAUSE: Agent's communication style, not functionality

💡 ACTIONABLE INSIGHTS:

1. Technical capability works (tool usage perfect)
2. Communication needs improvement (response quality failed)
3. Fix: Update agent instructions for clearer language or constrained response.



Día 4 (Calidad de agente)

Bienvenidos al Día 4.

Este documento técnico aborda el reto de garantizar la calidad en los agentes de AI mediante la introducción de un marco de evaluación holístico. La base técnica necesaria para esto es la Observabilidad, basada en tres pilares: Registros (el diario), Trazas (la narrativa) y Métricas (el informe de salud), que permite un bucle continuo de retroalimentación mediante métodos escalables como LLM-como-juez y evaluación Human-in-the-Loop (HITL).

En los laboratorios de código, aprenderás a usar logs, trazas y métricas para obtener visibilidad completa del proceso de toma de decisiones de tu agente, permitiéndote depurar fallos y entender por qué tu agente se comporta como lo hace. En el segundo codelab, aprenderás a evaluar a tus agentes para puntuar la calidad de respuesta y el uso de herramientas.

Día 4 Asignación

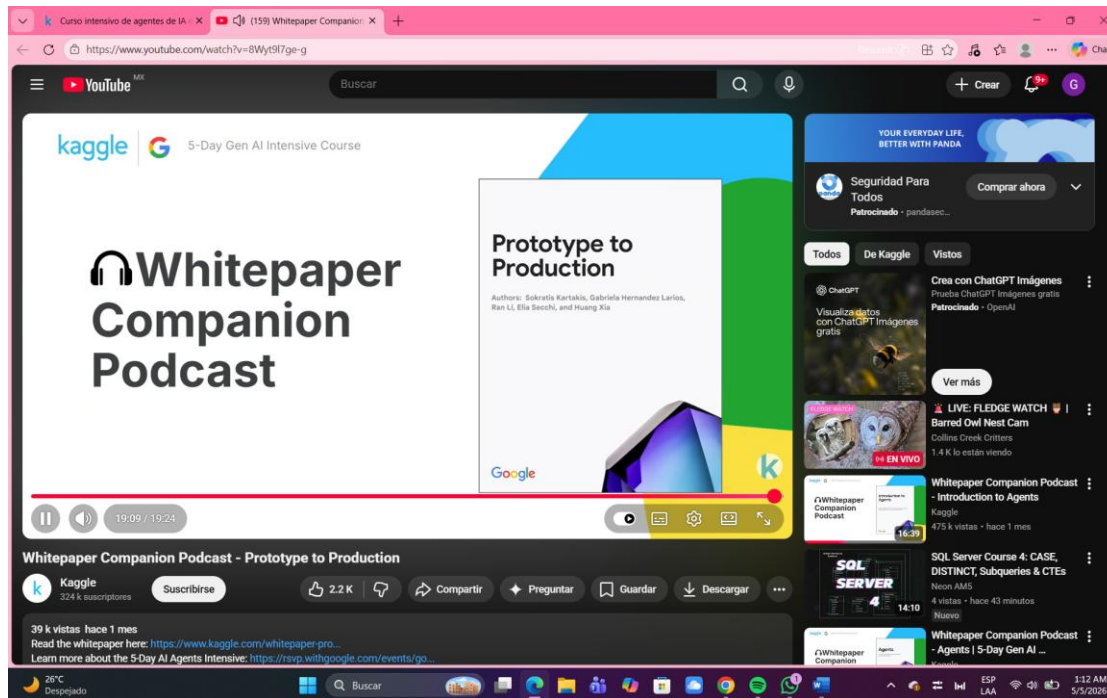
Completa Unidad 4 - "Calidad del Agente":

- ✓ Escucha el [episodio resumen del podcast](#) de esta unidad.
- ✓ Para complementar el pódcast, lee el [whitepaper de Calidad de Agente](#).
- ✓ Completa estos codelabs en Kaggle:
 - ✓ [Implementa](#) observabilidad para ayudarte a depurar tus agentes.
 - ✓ [Evalúa](#) a tus agentes.

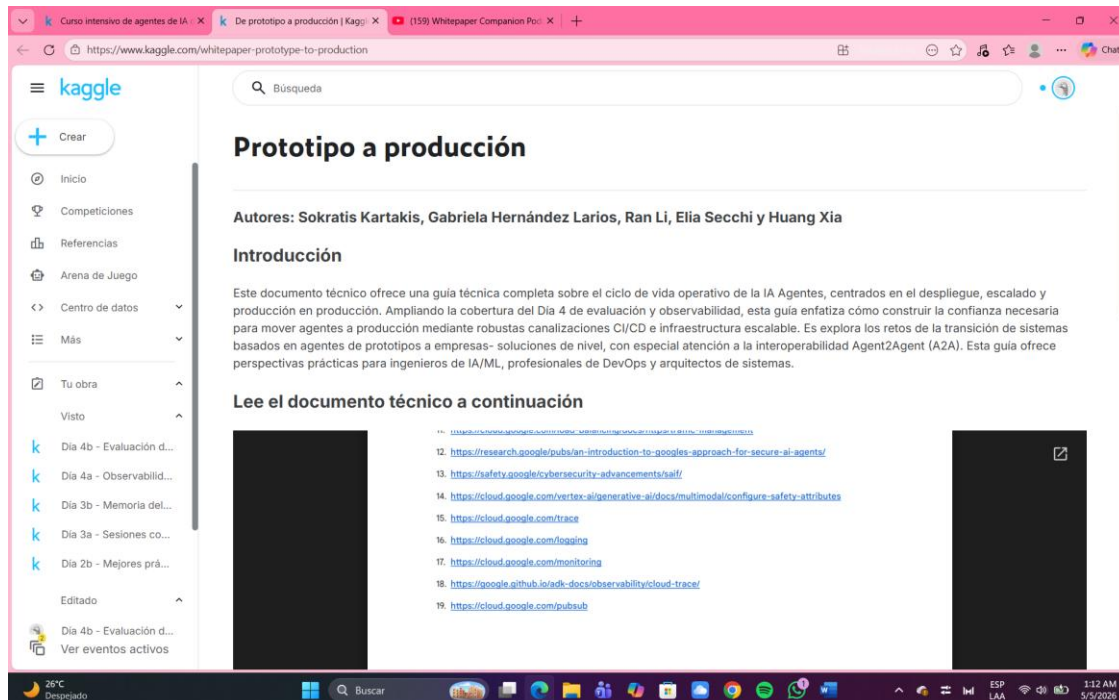
[Opcional]: Mira la [retransmisión en directo grabada de YouTube](#). Kanchana Patlolla y Anant Nawalgaria estuvieron acompañados por la autora de codelab, Sita Lakshmi Sangameswaran, junto con invitados de Google: Wafae Bakkali, Turan Bulmus y Sian Gooding, y la invitada externa Jiwei Liu de [NVIDIA](#).

Día 5 (Prototipo a Producción)

Escucha el episodio resumen del podcast de esta unidad.



Para complementar el pódcast, lee el whitepaper "Prototipo a Producción".



Explora cómo usar el Protocolo A2A para que los agentes interactúen entre sí.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Setup and authentication complete.")
except Exception as e:
    print(f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}")
    )

✅ Setup and authentication complete.
```

+ Code + Markdown

Import ADK components

```
import json
import requests
import subprocess
import time
import uuid

from google.adk.agents import LlmAgent
from google.adk.agents.remote_a2a_agent import (
    RemoteA2aAgent,
    AGENT_CARD_WELL_KNOWN_PATH,
)

from google.adk.a2a.utils.agent_to_a2a import to_a2a
from google.adk.models.google_llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types

# Hide additional warnings in the notebook
import warnings

warnings.filterwarnings("ignore")

print("✅ ADK components imported successfully.")

✅ ADK components imported successfully.
```

Curso intensivo de agentes de IA · Día 5a - Comunicación con Agentes · Kaggle Notebook Editor · De prototipo a producción | Kaggle · (159) Whitepaper Companion Po...

https://www.kaggle.com/code/fernandaximena/day-5a-agent2agent-communication/edit

Day 5a - Agent2Agent Communication

Archivo Edición Vista Corre Escenarios Complementos Ayuda

Compartir Guardar la versión 0

Sesión de borrador (3m)

```
product_lower = product_name.lower().strip()

if product_lower in product_catalog:
    return f"Product: {product_catalog[product_lower]}"
else:
    available = ", ".join([p.title() for p in product_catalog.keys()])
    return f"Sorry, I don't have information for {product_name}. Available products: {available}"

# Create the Product Catalog Agent
# This agent specializes in providing product information from the vendor's catalog
product_catalog_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="product_catalog_agent",
    description="External vendor's product catalog agent that provides product information and availability.",
    instructions="""
You are a product catalog specialist from an external vendor.
When asked about products, use the get_product_info tool to fetch data from the catalog.
Provide clear, accurate product information including price, availability, and specs.
If asked about multiple products, look up each one.
Be professional and helpful.
""",
    tools=[get_product_info], # Register the product lookup tool
)

print("✅ Product Catalog Agent created successfully!")
print("Model: gemini-2.5-flash-lite")
print("Tool: get_product_info()")
print("Ready to be exposed via A2A...")

✅ Product Catalog Agent created successfully!
Model: gemini-2.5-flash-lite
Tool: get_product_info()
Ready to be exposed via A2A...
```

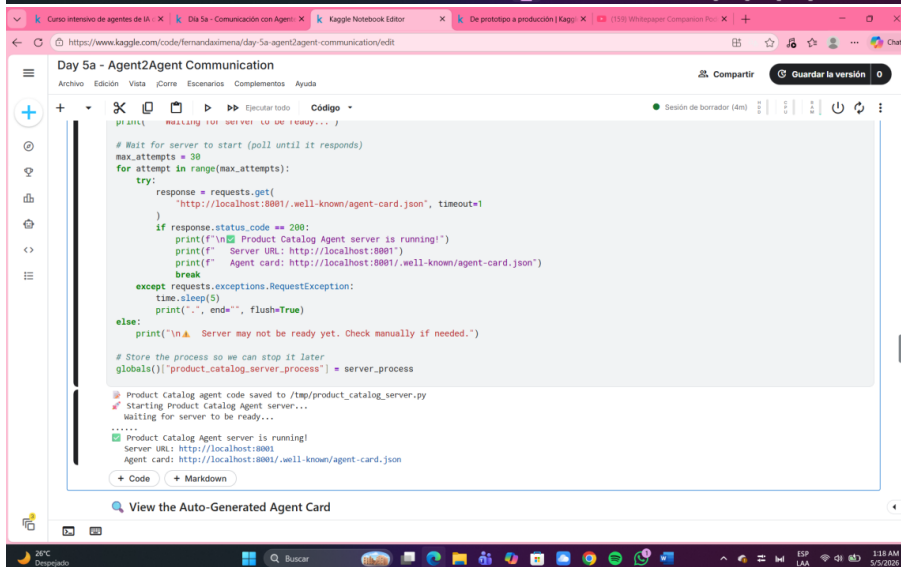
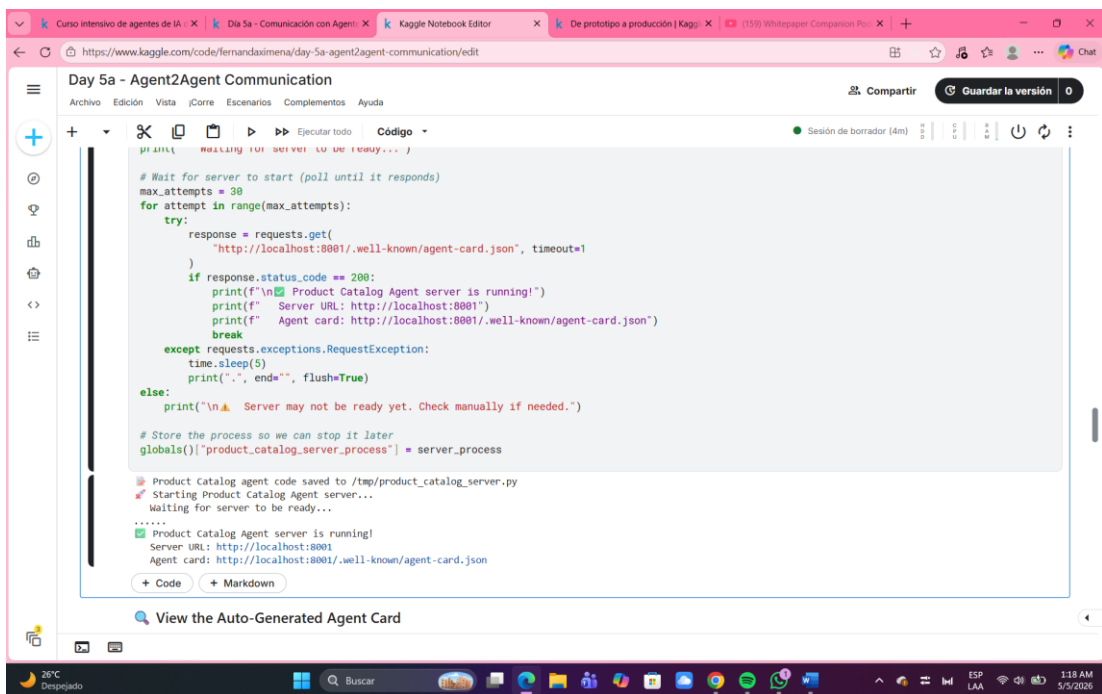
26°C Despejado 11:17 AM 5/5/2026

```
# Convert the product catalog agent to an A2A-compatible application
# This creates a FastAPI/Starlette app that:
# 1. Serves the agent at the A2A protocol endpoints
# 2. Provides an auto-generated agent card
# 3. Handles A2A communication protocol
product_catalog_a2a_app = to_a2a(
    product_catalog_agent, port=8001 # Port where this agent will be served
)

print("✅ Product Catalog Agent is now A2A-compatible!")
print("Agent will be served at: http://localhost:8001")
print("Agent card will be at: http://localhost:8001/.well-known/agent-card.json")
print("Ready to start the server...")

✅ Product Catalog Agent is now A2A-compatible!
Agent will be served at: http://localhost:8001
Agent card will be at: http://localhost:8001/.well-known/agent-card.json
Ready to start the server...
```

+ Code + Markdown



Curso intensivo de agentes de IA x Día 5a - Comunicación con Agentes x Kaggle Notebook Editor x De prototipo a producción | Kaggle x (159) Whisperer Companion Po x +

https://www.kaggle.com/code/fernandaximena/day-5a-agent2agent-communication/edit

Day 5a - Agent2Agent CommunicationDraft guardado

ArchivoEdiciónVistaCorreEscenariosComplementosAyuda

CompartirGuardar la versión0

Ejecutar todoCódigo

Sesión de borrador (5m)

Fetch the agent card from the running server

```
try:
    response = requests.get(
        "http://localhost:8001/.well-known/agent-card.json", timeout=5
    )

    if response.status_code == 200:
        agent_card = response.json()
        print("📄 Product Catalog Agent Card:")
        print(json.dumps(agent_card, indent=2))

        print("\n🔑 Key Information:")
        print(f"   Name: {agent_card.get('name')}")
        print(f"   Description: {agent_card.get('description')}")
        print(f"   URL: {agent_card.get('url')}")
        print(f"   Skills: {len(agent_card.get('skills', []))} capabilities exposed")
    else:
        print(f"❌ Failed to fetch agent card: {response.status_code}")

except requests.exceptions.RequestException as e:
    print(f"❌ Error fetching agent card: {e}")
    print("🔧 Make sure the Product Catalog Agent server is running (previous cell)")
```

```
{
  "capabilities": {},
  "defaultInputModes": [
    "text/plain"
  ],
  "defaultOutputModes": [
    "text/plain"
  ],
  "description": "External vendor's product catalog agent that provides product information and availability.",
  "name": "product_catalog_agent",
  "preferredTransport": "JSONRPC",
  "protocolVersion": "0.3.0",
  "skills": [
    {
      "description": "External vendor's product catalog agent that provides product information and availability. \n I am a product catalog specialist from an external vendor.\n When asked about products, use the get_product_info tool to fetch data from the catalog.\n Provide clear, accurate product information including price, availability, and specs.\n If asked about multiple products, look up each one.\n Be professional and helpful.\n ",
      "id": "product_catalog_agent",
      "name": "model",
      "tags": [
        "llm"
      ]
    },
    {
      "description": "Get product information for a given product.",
      "id": "product_catalog_agent-get_product_info",
      "name": "get_product_info",
      "tags": [
        "llm",
        "tools"
      ]
    }
  ],
  "supportsAuthenticatedExtendedCard": false,
  "url": "http://localhost:8001",
  "version": "0.0.1"
}
```

Product Catalog Agent Card:

```
{
  "capabilities": {},
  "defaultInputModes": [
    "text/plain"
  ],
  "defaultOutputModes": [
    "text/plain"
  ],
  "description": "External vendor's product catalog agent that provides product information and availability.",
  "name": "product_catalog_agent",
  "preferredTransport": "JSONRPC",
  "protocolVersion": "0.3.0",
  "skills": [
    {
      "description": "External vendor's product catalog agent that provides product information and availability. \n I am a product catalog specialist from an external vendor.\n When asked about products, use the get_product_info tool to fetch data from the catalog.\n Provide clear, accurate product information including price, availability, and specs.\n If asked about multiple products, look up each one.\n Be professional and helpful.\n ",
      "id": "product_catalog_agent",
      "name": "model",
      "tags": [
        "llm"
      ]
    },
    {
      "description": "Get product information for a given product.",
      "id": "product_catalog_agent-get_product_info",
      "name": "get_product_info",
      "tags": [
        "llm",
        "tools"
      ]
    }
  ],
  "supportsAuthenticatedExtendedCard": false,
  "url": "http://localhost:8001",
  "version": "0.0.1"
}
```

🔑 Key Information:
Name: product_catalog_agent
Description: External vendor's product catalog agent that provides product information and availability.
URL: http://localhost:8001
Skills: 2 capabilities exposed

+ Code + Markdown

26°C Despejado 11:19 AM 5/5/2026

```
"description": "External vendor's product catalog agent that provides product information and availability.",
"name": "product_catalog_agent",
"preferredTransport": "JSONRPC",
"protocolVersion": "0.3.0",
"skills": [
  {
    "description": "External vendor's product catalog agent that provides product information and availability. \n I am a product catalog specialist from an external vendor.\n When asked about products, use the get_product_info tool to fetch data from the catalog.\n Provide clear, accurate product information including price, availability, and specs.\n If asked about multiple products, look up each one.\n Be professional and helpful.\n ",
    "id": "product_catalog_agent",
    "name": "model",
    "tags": [
      "llm"
    ]
  },
  {
    "description": "Get product information for a given product.",
    "id": "product_catalog_agent-get_product_info",
    "name": "get_product_info",
    "tags": [
      "llm",
      "tools"
    ]
  }
],
"supportsAuthenticatedExtendedCard": false,
"url": "http://localhost:8001",
"version": "0.0.1"
}

🔑 Key Information:
Name: product_catalog_agent
Description: External vendor's product catalog agent that provides product information and availability.
URL: http://localhost:8001
Skills: 2 capabilities exposed
```

+ Code + Markdown

```
# Create a RemoteA2aAgent that connects to our Product Catalog Agent
# This acts as a client-side proxy - the Customer Support Agent can use it like a local agent
remote_product_catalog_agent = RemoteA2aAgent(
    name="product_catalog_agent",
    description="Remote product catalog agent from external vendor that provides product information.",
    # Point to the agent card URL - this is where the A2A protocol metadata lives
    agent_card=f"http://localhost:8001{AGENT_CARD_WELL_KNOWN_PATH}",
)

print("✅ Remote Product Catalog Agent proxy created!")
print(f"Connected to: http://localhost:8001")
print(f"Agent card: http://localhost:8001{AGENT_CARD_WELL_KNOWN_PATH}")
print("The Customer Support Agent can now use this like a local sub-agent!")

✅ Remote Product Catalog Agent proxy created!
Connected to: http://localhost:8001
Agent card: http://localhost:8001/.well-known/agent-card.json
The Customer Support Agent can now use this like a local sub-agent!
```

```
# Now create the Customer Support Agent that uses the remote Product Catalog Agent
customer_support_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="customer_support_agent",
    description="A customer support assistant that helps customers with product inquiries and information.",
    instruction="""
    You are a friendly and professional customer support agent.

    When customers ask about products:
    1. Use the product_catalog_agent sub-agent to look up product information
    2. Provide clear answers about pricing, availability, and specifications
    3. If a product is out of stock, mention the expected availability
    4. Be helpful and professional!

    Always get product information from the product_catalog_agent before answering customer questions.
    """,
    sub_agents=[remote_product_catalog_agent], # Add the remote agent as a sub-agent!
)

print("✅ Customer Support Agent created!")
print("Model: gemini-2.5-flash-lite")
print("Sub-agents: 1 (remote Product Catalog Agent via A2A)")
print("Ready to help customers!")

✅ Customer Support Agent created!
Model: gemini-2.5-flash-lite
Sub-agents: 1 (remote Product Catalog Agent via A2A)
Ready to help customers!
```

```
print("-" * 60)

# Run the agent asynchronously (handles streaming responses and A2A communication)
async for event in runner.run_async(
    user_id=user_id, session_id=session_id, new_message=test_content
):
    # Print final response only (skip intermediate events)
    if event.is_final_response() and event.content:
        for part in event.content.parts:
            if hasattr(part, "text"):
                print(part.text)

print("-" * 60)

# Run the test
print("🚀 Testing A2A Communication...\n")
await test_a2a_communication("Can you tell me about the iPhone 15 Pro? Is it in stock?")

🚀 Testing A2A Communication...

👤 Customer: Can you tell me about the iPhone 15 Pro? Is it in stock?

🗨️ Support Agent response:
-----
INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLLMVariant.GEMINI_API, stream: False
INFO:google_adk.google.adk.models.google_llm:Response received from the model.
WARNING:google_genai.types.warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full cand
idates.content.parts accessor to get the full model response.
INFO:google_adk.google.adk.agents.remote_a2a_agent:Successfully resolved remote A2A agent: product_catalog_agent
The iPhone 15 Pro is currently in stock, with only 8 units available. It is priced at $999 and features a 128GB storage capacity with a titanium finish.
-----
```

```
# Test comparing multiple products
await test_a2a_communication(
    "I'm looking for a laptop. Can you compare the Dell XPS 15 and MacBook Pro 14 for me?"
)

INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLMVariant.GEMINI_API, stream: False

Customer: I'm looking for a laptop. Can you compare the Dell XPS 15 and MacBook Pro 14 for me?

Support Agent response:
-----
INFO:google_adk.google.adk.models.google_llm:Response received from the model.
WARNING:google_genai.types.warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full and
dates.content.parts accessor to get the full model response.
The Dell XPS 15 is available for $1,299 and has 45 units in stock. It features a 15.6" display, 16GB RAM, and 512GB SSD. The MacBook Pro 14" is priced at $1,999 with 22
units in stock. It comes with an M3 Pro chip, 18GB RAM, and 512GB SSD.
```

```
# Test specific product inquiry
await test_a2a_communication(
    "Do you have the Sony WH-1000XM5 headphones? What's the price?"
)

INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLMVariant.GEMINI_API, stream: False

Customer: Do you have the Sony WH-1000XM5 headphones? What's the price?

Support Agent response:
-----
INFO:google_adk.google.adk.models.google_llm:Response received from the model.
WARNING:google_genai.types.warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full and
dates.content.parts accessor to get the full model response.
The Sony WH-1000XM5 headphones are available for $399. We have 67 units in stock. They feature noise-canceling technology and a 30-hour battery life.
```

[Opcional] Despliega tu agente en Agent Engine en Google Cloud.

1.2: Import components

Now, import the specific components you'll need for this notebook. This keeps your code organized and ensures we have access to the necessary building blocks.

```
import os
import random
import time
import vertexai
from kaggle_secrets import UserSecretsClient
from vertexai import agent_engines

print("✅ Imports completed successfully")

✅ Imports completed successfully
```

```

> # Set up Cloud Credentials in Kaggle
user_secrets = UserSecretsClient()
user_credential = user_secrets.get_gcloud_credential()
user_secrets.set_tensorflow_credential(user_credential)

print("✅ Cloud credentials configured")

```

✅ Cloud credentials configured

+ Code + Markdown

```

> ## Set your PROJECT_ID
PROJECT_ID = "your-project-id" # TODO: Replace with your project ID
os.environ["GOOGLE_CLOUD_PROJECT"] = PROJECT_ID

if PROJECT_ID == "ml-projecto-123" or not PROJECT_ID:
    raise ValueError("⚠️ Please replace 'your-project-id' with your actual Google Cloud Project ID.")

print(f"✅ Project ID set to: {PROJECT_ID}")

```

✅ Project ID set to: your-project-id

+ Code + Markdown

1.5: Enable Google Cloud APIs

```

> ## Create simple agent - all code for the agent will live in this directory
!mkdir -p sample_agent

print(f"✅ Sample Agent directory created")

```

✅ Sample Agent directory created

+ Code + Markdown

2.2: Create requirements file

```

> %%writefile sample_agent/requirements.txt

google-adk
opentelemetry-instrumentation-google-genai

```

Writing sample_agent/requirements.txt

+ Code + Markdown

2.3: Create environment configuration

We need to provide the agent with the necessary cloud configuration settings.

3.2: Select deployment region

Agent Engine is available in specific regions. We'll randomly select one for this demo.

```
regions_list = ["europe-west1", "europe-west4", "us-east4", "us-west1"]
dice = random.choice(regions_list)

print(f"Selected deployment region: {deployed_region}")
```

Selected deployment region: us-east4

+ Code + Markdown

3.3: Deploy the agent

This uses the ADK CLI to deploy your agent to Agent Engine.

```
!adk deploy agent_engine --project="mi-proyecto-123456" --region=$deployed_region sample_agent...
```

Usage: adk deploy agent_engine [OPTIONS] AGENT
Try 'adk deploy agent_engine --help' for help.

Error: Invalid value for 'AGENT': Directory 'sample_agent...' does not exist.

+ Code + Markdown

What just happened:

4.1: Retrieve the deployed agent

After deploying with the CLI, we need to retrieve the agent object to interact with it.

```
# Initialize Vertex AI
vertexai.init(project="mi-proyecto-123456", location=deployed_region)

# Get the most recently deployed agent
agents_list = list(agent_engines.list())
if agents_list:
    remote_agent = agents_list[0] # Get the first (most recent) agent
    client = agent_engines
    print(f"Connected to deployed agent: {remote_agent.resource_name}")
else:
    print("No agents found. Please deploy first.")
```

```
.....
InactiveRpcError          Traceback (most recent call last)
/usr/local/lib/python3.11/dist-packages/google/api_core/grpc_helpers.py in error_remapped_callable(*args, **kwargs)
    74         try:
--> 75             return callable(*args, **kwargs)
    76         except grpc.RpcError as exc:

/usr/local/lib/python3.11/dist-packages/grpc/_interceptor.py in __call__(self, request, timeout, metadata, credentials, wait_for_ready, compression)
    276         ) -> Any:
--> 277             response, ignored_call = self._with_call(
    278                 request,
331         )
```

```
32]: async for item in remote_agent.async_stream_query(
    message="What is the weather in Tokyo?",
    user_id="user_42",
):
    print(item)
```

```
.....
NameError                  Traceback (most recent call last)
/tmp/ipykernel_49/1869580144.py in <cell line: 1>()
----> 1 async for item in remote_agent.async_stream_query(
      2     message="What is the weather in Tokyo?",
      3     user_id="user_42",
      4 ):
      5     print(item)

NameError: name 'remote_agent' is not defined
```



Día 5 (Prototipo a Producción)

Bienvenidos al Día 5.

Este whitepaper ofrece una guía técnica sobre el ciclo de vida operativo de los agentes de IA, centrándose en el despliegue, escalamiento y producción. Explora los retos de la transición de sistemas agentivos de prototipos a soluciones de nivel empresarial, con especial atención al Protocolo Agent2Agent (A2A).

En los codelabs, aprenderás a construir sistemas de múltiples agentes independientes que puedan comunicarse y colaborar usando el Protocolo A2A. También aprenderás a llevar tu agente de tu máquina local a un servicio escalable listo para producción, desplegando tu agente en Vertex AI Agent Engine en Google Cloud.

Asignación

final Unidad 5 - "Prototipo a producción":

- ✓ Escucha el [episodio resumen del podcast](#) de esta unidad.
- ✓ Para complementar el pódcast, lee el [whitepaper "Prototipo a Producción"](#).
- ✓ Completa estos codelabs en Kaggle:
 - ✓ [Explora](#) cómo usar el Protocolo A2A para que los agentes interactúen entre sí.
 - ✓ [Opcional] [Despliega](#) tu agente en Agent Engine en Google Cloud.

[Opcional]: Mira la [retransmisión en directo grabada de YouTube](#). Kanchana Patlolla y Anant Nawalgaria estuvieron acompañados por la autora de Codelab, Laxmi Harikumar, junto con invitados de Google: Will Grannis, Sokratis Kartakis, Elia Secchi y Saurabh Tiwary.